

25 Performance and Optimization Plan

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

25 Performance and Optimization Plan

6

title: "Cartridge & Cloud — Performance and Optimization Plan" subtitle: "Plan maestro consolidado de rendimiento, profiling, memoria, carga, renderizado, escalabilidad y gates técnicos" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_number: "30" document_version: "2.0" project_version_reference: "0.0.26" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE"

30 — Performance and Optimization Plan	6
0. Propósito, autoridad y forma de uso	7
0.1. Propósito	7
0.2. Autoridad documental	8
0.3. Estados utilizados	8
0.4. Unidad de trabajo: Performance Requirement	9
0.5. Regla de medición antes de optimización	9
0.6. Qué no demuestra rendimiento	9
0.7. Revisión del documento	9
1. Genealogía histórica completa	10
1.1. Origen distribuido	10
1.2. Baseline v0.3 — intención técnica extensa	10
1.3. Baseline v0.4 — maduración arquitectónica	11
1.4. Baseline v0.5 — fotografía tras Sprint 5	11
1.5. Baseline v0.6 — estado previo a Sprint 16/17	11
1.6. Documentación consolidada 00–29	11
1.7. Decisiones históricas consolidadas	12
2. Fotografía técnica observada	12
2.1. Stack y paquetes	12
2.2. Dimensión del proyecto	12
2.3. Código y tests	13
2.4. Escenas	13
2.5. Build Profile observado	13
2.6. Player Settings observados	14
2.7. Quality y URP observados	14
2.8. Activos representativos	14
2.9. Puntos de auditoría de código	15
2.10. Evidencia conocida y desconocida	15
3. Principios de rendimiento	15
3.1. Frame time antes que FPS medio	15
3.2. La experiencia percibida es el resultado	16
3.3. Determinismo protegido	16
3.4. Optimización proporcional al alcance	16
3.5. Calidad escalable, identidad estable	16
3.6. Warm-up controlado	16
3.7. Cero coste invisible injustificado	17
3.8. Presupuesto por sistema	17
3.9. Regresión relativa y target absoluto	17
3.10. Evidencia durable	17
4. Objetivos de producto y presupuestos maestros	17
4.1. Objetivo del vertical slice	17
4.2. Presupuesto maestro provisional	18
4.3. Frame pacing provisional para H6	18
4.4. Presupuesto de CPU/GPU	18
4.5. Presupuesto de memoria provisional	18
4.6. Presupuesto de carga y respuesta	19
4.7. Presupuesto de estabilidad	19

4.8. Presupuesto de build y almacenamiento	19
4.9. Target de 30 FPS	20
5. Hardware, sistema operativo y entornos	20
5.1. No publicar hardware antes de medir	20
5.2. Perfiles de hardware requeridos	20
5.3. Campos obligatorios del registro	20
5.4. Matriz de sistema operativo	21
5.5. Almacenamiento	21
5.6. Temperatura y rendimiento sostenido	21
5.7. Refresh rate y sincronización	21
6. Metodología de profiling	21
6.1. Build types	21
6.2. Condiciones de una captura válida	22
6.3. Warm-up	22
6.4. Repeticiones	22
6.5. Herramientas Unity	22
6.6. Métricas mínimas por captura	22
6.7. Determinación de cuello de botella	23
6.8. Marcadores recomendados	23
6.9. Baseline storage	23
7. Catálogo de escenarios de rendimiento	24
7.1. PERF-SCN-001 — Cold Boot	24
7.2. PERF-SCN-002 — MainMenu estable	24
7.3. PERF-SCN-003 — Nueva partida	24
7.4. PERF-SCN-004 — StoreInitial vacía	24
7.5. PERF-SCN-005 — StoreInitial equipada	24
7.6. PERF-SCN-006 — Ocho clientes	24
7.7. PERF-SCN-007 — Pico de checkout	25
7.8. PERF-SCN-008 — Build Mode intensivo	25
7.9. PERF-SCN-009 — Operations/UI pesada	25
7.10. PERF-SCN-010 — Pedido y recepción	25
7.11. PERF-SCN-011 — Cierre de día	25
7.12. PERF-SCN-012 — Save manual/autosave	25
7.13. PERF-SCN-013 — Load de slot	25
7.14. PERF-SCN-014 — Siete días	25
7.15. PERF-SCN-015 — Scene lifecycle	25
7.16. PERF-SCN-016 — Stress progresivo	25
7.17. PERF-SCN-017 — Accesibilidad extrema	26
7.18. PERF-SCN-018 — Calidad gráfica	26
7.19. PERF-SCN-019 — Pérdida de foco	26
7.20. PERF-SCN-020 — Sesión térmica	26
8. CPU, simulación y scheduling	26
8.1. Rutas críticas	26
8.2. Update por entidad	26
8.3. Frecuencias recomendadas	27
8.4. ResolveReferences	27
8.5. Búsquedas y colecciones	27
8.6. Strings y logs	27
8.7. Physics y raycasts	27
8.8. Jobs/Burst/DOTS	28
9. Memoria, GC y lifecycle	28
9.1. Tipos de memoria a observar	28
9.2. Baseline de memoria	28
9.3. Leak vs cache	28
9.4. Allocations por frame	29
9.5. GC incremental	29
9.6. Pooling	29
9.7. Materiales y memoria	29
9.8. Save size	30

10. Renderizado y URP	30
10.1. Orden de investigación GPU	30
10.2. Resolución y render scale	30
10.3. Sombras	30
10.4. Luces	31
10.5. SSAO	31
10.6. Depth y Opaque Texture	31
10.7. Transparencias	31
10.8. SRP Batch y materiales	32
10.9. Dynamic batching y GPU Resident Drawer	32
10.10. Postproceso	32
10.11. Shader compilation y variants	32
11. Arte, geometría, materiales y contenido visual	32
11.1. Presupuestos heredados de Art Bible	32
11.2. LOD	33
11.3. Culling	33
11.4. Productos visibles	33
11.5. Texturas	33
11.6. Colliders	34
11.7. Animación	34
11.8. VFX	34
12. UI, texto y localización	34
12.1. UI como ruta crítica	34
12.2. Canvas	35
12.3. HUD	35
12.4. Listas	35
12.5. Texto y fuentes	35
12.6. Estados de carga	35
12.7. Input y latencia UI	35
13. Clientes, navegación e IA	36
13.1. Objetivo de población	36
13.2. Costes a separar	36
13.3. Pathfinding	36
13.4. Spawning	36
13.5. Tick escalonado	36
13.6. Stress y degradación segura	37
14. Carga de escenas, composición y lifecycle	37
14.1. Scene flow	37
14.2. StoreInitial	37
14.3. Runtime builders	37
14.4. Catálogos	38
14.5. Resources y carga directa	38
14.6. Duplicación	38
15. Persistencia, guardado y carga	38
15.1. Rendimiento no compromete integridad	38
15.2. Fases de save	38
15.3. Fases de load	39
15.4. Hilos y asincronía	39
15.5. Autosave	39
15.6. Crecimiento del save	39
16. Audio y rendimiento	39
16.1. Estado observado	39
16.2. Presupuesto de voces	40
16.3. Load type	40
16.4. Spam y cooldown	40
16.5. Profiling	40
17. Input, cámara, oclusión y accesibilidad	41
17.1. Independencia del frame rate	41
17.2. Cámara	41

17.3. Wall occlusion	41
17.4. Reducción de movimiento	41
17.5. Escalado de UI/texto	41
17.6. Opciones de rendimiento accesibles	41
18. Quality Settings, frame cap y opciones gráficas	42
18.1. Problema actual	42
18.2. Presets propuestos	42
18.3. Opciones candidatas	42
18.4. VSync y frame cap	42
18.5. Resolución y ventana	43
18.6. Auto-detección	43
19. Build, compilación y configuración de release	43
19.1. Perfiles necesarios	43
19.2. StoreInitial y TestLab	43
19.3. Development overhead	43
19.4. Logs	43
19.5. Stripping	44
19.6. Compresión de build	44
19.7. Reproducibilidad	44
20. Escalabilidad y sistemas futuros	44
20.1. Regla de reapertura de presupuestos	44
20.2. Empleados	44
20.3. Expansión física	45
20.4. Servicios online del juego	45
20.5. Steam Deck	45
21. Flujo de optimización y control de cambios	45
21.1. Intake	45
21.2. Clasificación	46
21.3. Hipótesis	46
21.4. Cambio mínimo	46
21.5. Verificación	46
21.6. Reversión	46
21.7. Deuda aceptada	47
22. QA de rendimiento y regresión	47
22.1. Severidades	47
22.2. Regresión significativa	47
22.3. Pruebas automatizadas	47
22.4. Pruebas manuales	48
22.5. Player.log	48
22.6. Informe de rendimiento	48
23. Gates por fase	48
23.1. Sprint 16	48
23.2. Sprint 17	49
23.3. H6	49
23.4. Demo/Playtest	49
23.5. Alpha/Beta	49
23.6. Release Candidate	50
23.7. Postlanzamiento	50
24. Roles y responsabilidades	50
25. Registro inicial de deuda y hallazgos	50
26. Work packages	51
PERF-WP-001 — Congelar benchmark de referencia	51
PERF-WP-002 — Registrar hardware DEV-REF	51
PERF-WP-003 — Crear Windows_Profiling	51
PERF-WP-004 — Crear Windows_ReleaseCandidate	51
PERF-WP-005 — Política de frame pacing	51
PERF-WP-006 — Opciones de resolución/ventana	51
PERF-WP-007 — Presets PC	51
PERF-WP-008 — Instrumentación base	52

PERF-WP-009 — Baseline CPU/GPU StoreInitial	52
PERF-WP-010 — Baseline memoria	52
PERF-WP-011 — Baseline carga	52
PERF-WP-012 — Baseline save	52
PERF-WP-013 — Auditoría Updates	52
PERF-WP-014 — Customer scheduling	52
PERF-WP-015 — Navegación	52
PERF-WP-016 — Placement	52
PERF-WP-017 — Inventory visual sync	52
PERF-WP-018 — UI profiling	52
PERF-WP-019 — Shadow study	52
PERF-WP-020 — SSAO study	52
PERF-WP-021 — Opaque/depth audit	52
PERF-WP-022 — Material/batch audit	53
PERF-WP-023 — LOD/culling validation	53
PERF-WP-024 — Texture budget	53
PERF-WP-025 — Character budget	53
PERF-WP-026 — VFX budget	53
PERF-WP-027 — Audio profiling	53
PERF-WP-028 — Scene lifecycle soak	53
PERF-WP-029 — Seven-day soak	53
PERF-WP-030 — Hardware mínimo candidato	53
PERF-WP-031 — Hardware recomendado candidato	53
PERF-WP-032 — Build size/shader report	53
PERF-WP-033 — Performance regression report	53
PERF-WP-034 — H6 Performance Report	53
PERF-WP-035 — Steam requirements approval	53
PERF-WP-036 — CI/performance smoke	54
PERF-WP-037 — Postlaunch diagnostics	54
27. Registro de riesgos	54
28. Playbooks operativos	54
28.1. Caída repentina de FPS	54
28.2. Stutter al abrir panel	55
28.3. Crecimiento de memoria	55
28.4. GPU bound	55
28.5. CPU bound por clientes	55
28.6. Save lento	55
28.7. Load lento	56
28.8. Shader hitch	56
28.9. Build peor que Editor o viceversa	56
28.10. Report de usuario	56
29. Plantillas de evidencia	56
29.1. Cabecera de benchmark	56
29.2. Resultado de escenario	57
29.3. Comparación antes/después	57
29.4. Deuda aceptada	57
30. Checklists	57
30.1. Antes de medir	58
30.2. CPU	58
30.3. GPU	58
30.4. Memoria	58
30.5. UI/accesibilidad	58
30.6. H6	59
31. Integración con otros documentos	59
31.1. Vertical Slice Specification	59
31.2. TDD	59
31.3. QA Plan/Matrix	59
31.4. Art Bible	59
31.5. Audio Bible	59

31.6. UI Style Guide y Accessibility	59
31.7. Build Guide	59
31.8. Steam Publishing y Marketing	60
31.9. Privacy y Security	60
31.10. Documentos 31-34	60
32. Fuentes históricas directas	60
33. Fuentes vigentes y artefactos inspeccionados	60
33.1. Documentación consolidada	60
33.2. Configuración inspeccionada	61
33.3. Registros históricos de desarrollo	61
33.4. Limitación de la fotografía	61
34. Criterios de cierre del documento 30	61
Actualización de rendimiento W0	62
Estado operativo vigente tras W0	62
Escenarios de caracterización obligatorios	62
Regla de cierre y no propagación	62

25 Performance and Optimization Plan

Metadatos normativos v2.0

document_version: 2.0

project_version: 0.0.26

last_reviewed: 2026-07-17

status: APPROVED LIVING PLAN / PENDING FUTURE MEASUREMENT

lifecycle: LIVING DOCUMENT

title: "Cartridge & Cloud — Performance and Optimization Plan" subtitle: "Plan maestro consolidado de rendimiento, profiling, memoria, carga, renderizado, escalabilidad y gates técnicos" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_number: "30" document_version: "2.0" project_version_reference: "0.0.26" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE"

30 — Performance and Optimization Plan

Proyecto: Cartridge & Cloud

Desarrollador: VRM Games / Blas Luis Rocha González

Plataforma inicial: PC / Steam, Windows x64

Estado técnico de referencia: Sprints 0-15 CLOSED / PASS; Sprint 16 COMPLETED / PASS; Sprint 17 APPROVED / IMPLEMENTED / VALIDATED / CLOSED; H6 y Release Candidate PENDING / NOT RUN

Objetivo histórico heredado: 60 FPS a 1920 × 1080, tienda equipada y hasta 8 clientes simultáneos

Presupuestos históricos heredados: carga del vertical slice < 5 s, cierre/guardado < 2 s, allocations por frame próximas a cero en simulación estable

Estado de medición: no existe todavía un informe de profiling de build representativa que permita aprobar hardware mínimo, percentiles, memoria máxima, requisitos de Steam o claims públicos de optimización

Clasificación: documento interno normativo para ingeniería, arte, UI, audio, QA, producción, build, accesibilidad, publicación y soporte

Naturaleza: plan de producto y proceso; no constituye un benchmark final, una promesa comercial ni una autorización para publicar requisitos mínimos

Regla principal: ningún cambio se considera optimización porque reduzca líneas de código, desactive una función o mejore una captura aislada. Debe resolver un coste reproducible, mantener las invariantes del producto, superar una comparación antes/después en la misma build y escenario, y no transferir el problema a accesibilidad, claridad, calidad visual, persistencia o determinismo.

Regla de evidencia: el Editor sirve para localizar problemas; la aceptación se decide en Player externo identificado. Un FPS medio sin frame time, percentiles, hardware, calidad, población y duración no es evidencia suficiente.

Regla de alcance: el objetivo de 60 FPS a 1080p se conserva como gate del vertical slice en el equipo de referencia. No se rebaja silenciosamente por falta de medición ni se extrapola a “cualquier PC”. Los requisitos mínimos y recomendados de Steam permanecen TBD hasta ejecutar la matriz definida en este plan.

Regla de protección del juego: rendimiento, determinismo, accesibilidad y legibilidad son restricciones simultáneas. No se puede mejorar frame time eliminando feedback crítico, reduciendo texto, rompiendo subtítulos, ocultando clientes relevantes, retrasando transacciones autoritativas o alterando resultados de simulación.

0. Propósito, autoridad y forma de uso

0.1. Propósito

Este documento consolida todos los requisitos de rendimiento y optimización localizados desde las baselines históricas v0.3-v0.6 hasta la documentación vigente 00-29 y la instantánea real del proyecto Unity anterior a la regeneración documental. Sus objetivos son:

1. preservar los presupuestos históricos completos y las decisiones técnicas que quedaron repartidas entre TDD, Art Bible, Audio Bible, UI, QA, roadmap, build y código;
2. convertir objetivos generales en escenarios, métricas, límites, gates y evidencias reproducibles;
3. separar target de producto, configuración observada, baseline medida, regresión, deuda aceptada y claim público;
4. impedir optimización prematura o basada únicamente en intuición;
5. fijar un lenguaje común para CPU, GPU, frame pacing, memoria, GC, carga, guardado, audio, UI, navegación y build;
6. establecer una matriz progresiva de hardware sin inventar requisitos mínimos antes de disponer de mediciones;
7. definir cómo se construye, captura, compara, archiva y aprueba un perfil de rendimiento;
8. integrar rendimiento con StoreInitial, arte representativo, ocho clientes, Golden Path, siete días, save/load, ES/EN y accesibilidad extrema;
9. documentar la configuración observada de Unity y distinguirla de la configuración final objetivo;
10. identificar riesgos y hotspots del código actual sin calificarlos como defectos antes de medir;
11. transformar el trabajo futuro en paquetes ejecutables, criterios de aceptación y owners;
12. preparar las entradas que utilizarán el checklist H6, el manifest de baseline, el registro global de riesgos y el manual final de operaciones.

0.2. Autoridad documental

La jerarquía aplicable es:

- 00_Enfoque_y_Alcance.md, para visión, pilares, alcance y límites del producto.
- 01_Game_Design_Document.md, para reglas de simulación, población, ritmo y sistemas.
- 02_Vertical_Slice_Specification.md, para el objetivo heredado de rendimiento y la definición de H6.
- 03_Technical_Design_Document.md, para presupuestos técnicos, arquitectura, memoria y prácticas prohibidas.
- 04_Modelo_de_Datos.md, para persistencia, snapshots, invariantes y tamaño de estado.
- 05_UX_Flow.md, para tiempos de respuesta, estados de carga, input y feedback.
- 06_Production_Roadmap_y_Sprint_Plan.md, para Sprint 17, workstreams, prioridades y gates.
- 07_QA_Testing_Plan.md y 08_QA_Testing_Matrix.xlsx, para metodología de prueba, severidades y evidencia.
- 09_CSharp_Coding_Standards.md, para loops calientes, allocations, lifecycle y profiling.
- 10_Unity_Project_Setup_Guide.md y 11_Build_y_Versioning_Guide.md, para configuración, Player, Build Profiles y reproducibilidad.
- 17_Art_Bible.md, para presupuestos artísticos, materiales, iluminación, LOD y densidad.
- 18_Audio_Bible.md, para voces, carga, compresión, streaming y Audio Profiler.
- 19_UI_Style_Guide.md, para Canvas, layout, escalado, actualización y estados.
- 22_Localization_Plan.md, para escenarios de expansión de texto y fuentes.
- 24_Steam_Publishing_Plan.md, para requisitos publicables, compatibilidad y Steam Deck cuando corresponda.
- 25_Post_Launch_and_Live_Operations_Plan.md, para regresiones, hotfix y soporte.
- 26_Privacy_Data_and_Telemetry_Plan.md, para métricas, diagnósticos y datos de hardware.
- 27_Security_and_Incident_Response_Plan.md, para builds de profiling, símbolos, logs y artefactos privados.
- 28_Marketing_and_Communication_Plan.md, para claims de “optimizado”, capturas y requisitos.
- 29_Accessibility_and_Inclusive_Design_Plan.md, para estabilidad perceptual, reducción de movimiento, escalado y opciones.
- Este documento, como autoridad especializada en rendimiento y optimización mientras no contradiga una fuente superior.

Cuando exista una contradicción, no se elige la cifra o configuración más favorable. Se registra un Change ID, se reproduce la discrepancia, se determina qué documento está desactualizado y se actualizan código, tests, criterios y claims afectados.

0.3. Estados utilizados

Estado	Significado
HISTORICAL	Decisión conservada por trazabilidad; no demuestra estado actual.
CURRENT / OBSERVED	Configuración o implementación presente en la instantánea inspeccionada.
BASELINE REQUIRED	Debe medirse antes de optimizar o aceptar una regresión.
TARGET	Objetivo de producto aprobado para el escenario indicado.
PROVISIONAL BUDGET	Límite de planificación sujeto a validación en hardware y build.
VERIFIED	Medido de forma reproducible y archivado.
PASS	Cumple el gate, sin defectos bloqueantes ni evidencia insuficiente.
FAIL	Incumple target o presenta degradación bloqueante.
ACCEPTED DEBT	Incumplimiento documentado, acotado, con owner, trigger y fecha de revisión.
TBD	No se dispone de información suficiente; no puede publicarse.
NOT OPEN	Sistema futuro que no debe consumir presupuesto de producción todavía.
SUPERSEDED	Decisión sustituida formalmente y conservada solo como antecedente.

0.4. Unidad de trabajo: Performance Requirement

Cada requisito o hallazgo de rendimiento debe incluir:

- ID estable;
- escenario;
- build ID, commit/SHA y versión;
- hardware, OS, driver, resolución y calidad;
- población, contenido y duración;
- métrica primaria y métricas auxiliares;
- baseline de comparación;
- umbral o criterio;
- resultado;
- capturas de Profiler, Memory Profiler o Frame Debugger cuando proceda;
- logs y marcadores;
- cambio responsable;
- impacto funcional, visual y de accesibilidad;
- owner;
- riesgo de regresión;
- decisión de cierre.

0.5. Regla de medición antes de optimización

El flujo obligatorio es:

síntoma reproducible
→ escenario congelado
→ baseline en build
→ clasificación CPU/GPU/I/O/memoria
→ hipótesis
→ cambio mínimo
→ misma medición
→ regresión funcional y visual
→ decisión

Se prohíbe aplicar varias optimizaciones simultáneas sin aislar su efecto, salvo respuesta urgente a una regresión donde se conserve una rama o serie de commits que permita atribución posterior.

0.6. Qué no demuestra rendimiento

No bastan por sí solos:

- FPS mostrado por Game View;
- una captura de Profiler de pocos segundos;
- una escena vacía;
- un resultado en Editor;
- ausencia de excepciones;
- bajo tamaño de un FBX;
- número pequeño de polígonos;
- compilación sin warnings;
- tests funcionales verdes;
- sensación subjetiva de fluidez en un equipo potente;
- desactivar VSync;
- reducir resolución sin registrar el cambio;
- medir una build Development y extrapolarla a Release o viceversa.

0.7. Revisión del documento

Debe revisarse:

- al cerrar Sprint 16;

- durante y al cerrar Sprint 17;
- antes de H6;
- cuando StoreInitial entre en el Build Profile;
- al cambiar Unity, URP, paquetes, render path o calidad;
- al integrar modelos de personaje, VFX, texturas finales o AudioMixer;
- al añadir empleados, más de ocho clientes o contenido de progresión;
- antes de demo, Playtest, festival, beta y Release Candidate;
- al publicar o modificar requisitos mínimos/recomendados;
- tras una regresión de rendimiento de severidad S0-S2;
- cuando una opción de accesibilidad tenga coste material.

1. Genealogía histórica completa

1.1. Origen distribuido

No se localizó un documento histórico autónomo denominado Performance Plan. El rendimiento estaba distribuido entre:

- TDD v0.3-v0.6;
- Vertical Slice Specification v0.1-v0.4;
- QA Testing Plan v0.3-v0.6;
- Roadmap v0.3-v0.6;
- Art Bible v0.3-v0.5;
- Audio Bible v0.3-v0.5;
- UI Style Guide v0.3-v0.5;
- C# Coding Standards v0.3-v0.5;
- Unity Project Setup Guide v0.3-v0.6;
- Build & Versioning Guide v0.3-v0.6;
- GDD, UX, Enfoque y Guía;
- ADR, charters, cierres y registros de Sprint;
- configuración y código del proyecto.

La ausencia de un título específico no implica ausencia de requisitos. Este documento preserva la especificación extensa de las primeras baselines y las fotografías técnicas más breves de las posteriores.

1.2. Baseline v0.3 — intención técnica extensa

La TDD v0.3 fijó por primera vez los presupuestos numéricos del vertical slice:

Métrica histórica	Objetivo v0.3
FPS a 1080p	60
Clientes simultáneos	8, escalable
Allocations por frame	Próximas a cero durante simulación estable
Tiempo de cierre/guardado	< 2 s en equipo de desarrollo
Tiempo de carga	< 5 s en vertical slice

La Art Bible v0.3 añadió atlas, materiales compartidos, LOD en props grandes, iluminación baked/mixta y representación agrupada de productos pequeños. La Audio Bible v0.3 introdujo pooling de fuentes y un servicio desacoplado. Los Coding Standards, QA y VSS históricos establecieron medición, estabilidad, build externa y sesiones prolongadas.

Estas decisiones siguen vigentes como origen. No se descartan porque una baseline posterior las resumiera.

1.3. Baseline v0.4 — maduración arquitectónica

La TDD v0.4 conservó los mismos presupuestos y amplió límites de arquitectura:

- IA mediante estados pequeños y movimiento delegado a NavMesh;
- tareas y reservas controladas;
- persistencia atómica;
- no rebakear navegación de forma indiscriminada;
- separar Domain de Unity;
- medir antes de introducir complejidad;
- evitar instanciación recurrente sin estrategia.

La VSS v0.2 y QA v0.4 reforzaron el uso de build, hardware registrado, smoke tests y evidencia. La repetición de las cifras en dos baselines demuestra continuidad, no duplicación accidental.

1.4. Baseline v0.5 — fotografía tras Sprint 5

La documentación v0.5 pasó de una especificación amplia a una fotografía operativa más breve:

- tienda técnica de 10 × 15 m;
- grid 20 × 30 de 0,5 m;
- arquitectura funcional todavía representada con primitivas;
- fundamentos de input, cámara, placement y acceso cerrados;
- Build Profile y escenas técnicas disponibles;
- pase representativo, profiling final y requisitos de release pendientes.

Esta baseline no sustituyó el objetivo de 60 FPS ni los tiempos de carga/guardado. Simplemente no los volvió a desarrollar con la misma profundidad.

1.5. Baseline v0.6 — estado previo a Sprint 16/17

La baseline v0.6 consolidó:

- Unity 6.3 LTS;
- URP;
- contenido representativo de arquitectura, muebles, productos, personajes y expansiones;
- StoreInitial como target de Sprint 16;
- Sprint 17 como fase de estabilización, balance, profiling y build;
- 60 FPS a 1080p con ocho clientes como objetivo heredado;
- ausencia de hardware mínimo aprobado;
- necesidad de validar StoreInitial, arte, audio y Player externo.

Las Art y Audio Bible v0.5 reconocieron que LOD, colliders, clips y assets presentes no equivalían a rendimiento aprobado. La VSS v0.4 mantuvo el gate sin inventar resultados.

1.6. Documentación consolidada 00-29

La nueva baseline desarrolló los requisitos históricos en documentos especializados:

- 02 conserva el objetivo de 60 FPS y exige hardware, percentiles, memoria y duración.
- 03 fija presupuestos, prácticas y escenarios de profiling.
- 06 reserva Sprint 17 para rendimiento y exige comparación antes/después.
- 07 define frame time, CPU, GPU, memoria, allocations, carga, save/load y soak.
- 09 limita trabajo por frame, LINQ, strings, búsquedas globales y materiales instanciados.
- 10 documenta URP, escenas, calidad y setup.
- 11 obliga a Player externo, Build ID, entorno y artefactos.
- 15 aporta procedimientos de regresión, profiling y cierre.
- 17 fija presupuestos artísticos, LOD, materiales, luces y densidad.
- 18 fija voces, carga, compresión, streaming y Audio Profiler.

- 19 cubre UI, layout, localización y escalado.
- 29 obliga a probar 30/60 FPS, reducción de movimiento, UI/texto 150 % y resolución mínima.

1.7. Decisiones históricas consolidadas

Decisión	Estado	Aplicación actual
60 FPS a 1080p	TARGET	Gate del vertical slice en hardware de referencia
8 clientes simultáneos	TARGET	Escenario representativo mínimo, no máximo futuro
Carga <5 s	PROVISIONAL BUDGET	Debe dividirse por arranque, escena y slot y medirse
Guardado/cierre <2 s	PROVISIONAL BUDGET	Debe incluir serialización, escritura y feedback
Allocations próximas a cero	TARGET	Rutas estables de simulación, UI e input
Atlas/materiales compartidos	TARGET	Aplicar cuando reduzca coste y mantenga authoring
LOD para props grandes	TARGET	Ya existe en parte del contenido; debe validarse
Iluminación baked/mixta	PROPOSED	Decidir tras composición final y perfilado
Productos agrupados	TARGET	Evitar una malla por unidad cuando no aporte lectura
Pooling de audio/VFX/clientes	CONDITIONAL	Solo con churn medido y lifecycle seguro
IA por eventos/ticks	TARGET	Evitar coste por agente y frame cuando no sea necesario
Build externa	REQUIRED	Editor no aprueba Sprint 17/H6
Medir antes de optimizar	REQUIRED	Toda intervención relevante
No sacrificar claridad	REQUIRED	Arte, UI, audio y accesibilidad

2. Fotografía técnica observada

2.1. Stack y paquetes

La instantánea inspeccionada usa:

Componente	Versión observada
Unity	6000.3.18f1
URP	17.3.0
Input System	1.19.0
AI Navigation	2.0.13
Localization	1.5.12
Test Framework	1.6.0
uGUI	2.0.0

No se actualizará un paquete como “optimización” incidental. Toda actualización requiere benchmark antes/después, revisión de shaders, escenas, input, navegación, builds, tests y compatibilidad.

2.2. Dimensión del proyecto

Excluyendo .meta, la carpeta Assets/_Project contiene aproximadamente:

Tipo	Cantidad observada	Tamaño aproximado
Archivos totales	768	11,19 MB
C#	457	2,53 MB / 81.780 líneas aprox.

Tipo	Cantidad observada	Tamaño aproximado
Prefabs	64	2,58 MB
FBX	62	4,85 MB
Materials	52	0,20 MB
Assets Unity	54	0,12 MB
PNG	12	0,10 MB
WAV	10	0,47 MB
Animaciones	11	0,03 MB
Escenas	5	0,25 MB

Estas cifras son pequeñas porque el contenido sigue siendo representativo y estilizado. No deben extrapolarse al tamaño final de build, memoria residente o VRAM.

2.3. Código y tests

La instantánea contiene:

Área	Archivos C#	Líneas aproximadas
Scripts de proyecto	300	44.119
Tests	143	27.408
Editor	14	10.253
Total	457	81.780

Se observan 9 métodos Update y 2 LateUpdate. La cantidad es razonable, pero varios métodos resuelven referencias, sincronizan vistas o actualizan agentes. Se clasificarán como **puntos de auditoría**, no como defectos automáticos.

2.4. Escenas

Escena	GameObjects serializados aprox.	MonoBehaviours aprox.	Estado relevante
Bootstrap	1	1	incluida en build
MainMenu	12	14	incluida en build
Store	33	21	incluida; escena histórica
StoreInitial	33	23	existe; no incluida todavía
TestLab	9	11	incluida en Development Profile

La igualdad aproximada de estructura entre Store y StoreInitial no demuestra que la segunda integre el pase representativo. La Art Bible vigente registra que StoreInitial todavía no instancia el conjunto representativo completo y que mantiene vínculos históricos.

2.5. Build Profile observado

Windows_Development:

- Development Build activada;
- profiler autoconnect desactivado;
- deep profiling desactivado;
- incluye Bootstrap, MainMenu, Store y TestLab;
- no incluye StoreInitial;
- no es un perfil de Release Candidate;

- su lista de escenas no debe modificarse hasta conectar y validar StoreInitial.

Un perfil Development no sirve para publicar requisitos. Debe existir una pareja de perfiles reproducible para profiling y release.

2.6. Player Settings observados

Ajuste	Valor observado	Interpretación
Resolución por defecto	1024 × 768	configuración técnica, no target comercial
Ventana redimensionable	no	debe revisarse para PC/Steam
Fullscreen mode serializado	1	validar comportamiento real
VSync	0 en perfiles de calidad	sin sincronización forzada
Límite de FPS explícito	no localizado	frame pacing pendiente
Incremental GC	activado	base válida, requiere profiling
Run in background	desactivado	coherente con foco; probar audio/pausa
Player log	activado	necesario para diagnóstico
Color space	Linear	apropiado para URP
Strip engine code	activado	validar builds y reflexión

La combinación VSync 0 + ausencia de `Application.targetFrameRate` puede producir FPS no limitados, consumo innecesario y comparaciones inconsistentes. Debe decidirse una política explícita.

2.7. Quality y URP observados

Existen perfiles Mobile y PC. El target actual es PC; Mobile no debe convertirse en promesa de plataforma.

El asset PC observado incluye:

- render scale 1.0;
- depth texture y opaque texture requeridas;
- HDR soportado;
- SRP Batcher activado;
- dynamic batching desactivado;
- main light y additional lights con sombras;
- hasta cuatro luces adicionales por objeto;
- shadow distance 50 en el asset URP;
- cuatro cascadas en el asset URP;
- soft shadows;
- SSAO activo;
- GPU Resident Drawer desactivado;
- streaming mipmaps desactivado;
- MSAA serializado como 1;
- postprocesos presentes pero con intensidades principales a cero en el perfil observado.

Estos ajustes son una baseline de configuración, no una configuración optimizada. Cada feature se evaluará por coste y contribución visual en StoreInitial.

2.8. Activos representativos

Se observan:

- 24 prefabs de arquitectura;
- 8 de mobiliario;

- 10 de productos/packaging;
- 3 de personajes;
- 19 conceptuales de expansión;
- LODGroup en muchas familias de muebles, productos y expansiones;
- 52 materiales;
- ausencia de texturas de producción y VFX finales en las rutas previstas;
- 10 clips WAV cortos;
- personajes todavía sin modelo final integrado.

El coste actual será menor que el coste de una escena con personajes, texturas, VFX y audio finales. El profiling de la instantánea sirve como baseline temprana, no como aprobación del release.

2.9. Puntos de auditoría de código

Se han localizado, entre otros:

- ResolveReferences() dentro de algunos Update de input;
- sincronización de placement y removals por frame;
- Update por agente de cliente;
- spawner y ciclo diario con tick por frame;
- actualización de HUD limitada a 0,25 s;
- wall occlusion en LateUpdate;
- instanciación de contenido representativo y stock;
- búsquedas globales en composición, fallbacks y tests;
- creación/destrucción de vistas de placement e inventario;
- acceso a Resources en catálogos y fallbacks;
- strings y logs en múltiples sistemas.

No se refactorizará por conteo estático. El Profiler determinará frecuencia, coste, allocations y prioridad.

2.10. Evidencia conocida y desconocida

Área	Conocido	Falta
Funcional	suites históricas verdes y Golden Path parcial	validación post-StoreInitial
Render	URP y assets configurados	GPU profile representativo
CPU	arquitectura y hotspots potenciales	captura de main thread en PI
Memoria	tamaño de proyecto y GC incremental	snapshots, pico y crecimiento
Carga	transición asíncrona documentada	tiempos medidos por escenas
Save/load	persistencia atómica implementada	latencia, tamaño y picos
Audio	10 clips y cuatro fuentes/canales	voces, memoria y streaming
Hardware	equipo de desarrollo no cumplimentado en registro histórico	matriz mínima/recomendada
Frame pacing	target histórico 60 FPS	política de VSync/cap y perce
Release	Windows Development profile	profiling y RC profiles

3. Principios de rendimiento

3.1. Frame time antes que FPS medio

A 60 FPS cada frame dispone de 16,67 ms; a 30 FPS, 33,33 ms. El promedio puede ocultar stutter. Se registrarán al menos mediana, P95, P99, máximo razonable, número de frames por encima de 33,33 ms y eventos asociados.

3.2. La experiencia percibida es el resultado

Un juego puede alcanzar 60 FPS y seguir sintiéndose mal por:

- spikes al abrir paneles;
- GC durante placement;
- camera jitter;
- input lag;
- hitch de shader;
- audio con cortes;
- carga sin feedback;
- save que congela;
- frame pacing irregular;
- transición de VSync o modo de ventana incorrecta.

El gate incluye fluidez, latencia y estabilidad, no solo throughput.

3.3. Determinismo protegido

No se optimizará alterando el resultado lógico según FPS. Dinero, inventario, reservas, checkout, ciclo diario y persistencia deben usar tiempo o eventos apropiados y producir resultados equivalentes dentro de las tolerancias diseñadas.

3.4. Optimización proporcional al alcance

El vertical slice requiere ocho clientes, no cientos. No se construirá una arquitectura masiva antes de demostrar necesidad. Sí se evitarán decisiones que impidan escalar razonablemente, como lógica completa por frame en cada entidad sin seam de scheduling.

3.5. Calidad escalable, identidad estable

Los niveles de calidad pueden reducir:

- sombras;
- SSAO;
- distancia de shadow/culling;
- densidad de VFX;
- calidad de postproceso;
- anisotropía;
- render scale cuando se autorice.

No deben eliminar:

- señalización crítica;
- feedback de error/éxito;
- legibilidad de producto;
- información de cola/checkout;
- alternativas visuales al audio;
- foco de UI;
- texto o subtítulos;
- elementos interactivos.

3.6. Warm-up controlado

Un coste que ocurre una vez puede ser aceptable si:

- se produce durante carga con feedback;
- no bloquea input sin explicación;
- no se repite tras cada panel o día;

- no supera el presupuesto del flujo;
- queda registrado.

No se “oculta” un hitch precalentando todo el proyecto y elevando memoria sin medir.

3.7. Cero coste invisible injustificado

Servicios desactivados, listeners duplicados, loops silenciosos, catálogos repetidos o GameObjects técnicos no deben consumir CPU por existir. El lifecycle debe desuscribir, desactivar y liberar.

3.8. Presupuesto por sistema

Cada sistema nuevo debe declarar:

- frecuencia de actualización;
- población máxima del hito;
- complejidad esperada;
- allocations;
- objetos visuales;
- impacto en save;
- carga inicial;
- peor caso;
- degradación segura.

3.9. Regresión relativa y target absoluto

Se usan ambos:

- **target absoluto:** 60 FPS/1080p, tiempos y estabilidad;
- **regresión relativa:** diferencia contra la baseline anterior en el mismo escenario.

Una build puede seguir por encima de 60 FPS y ser rechazada si introduce una regresión del 35 % sin causa aceptada. También puede mejorar 20 % y seguir fallando el target.

3.10. Evidencia durable

Capturas y reportes deben conservarse junto al Build ID. Un perfil sin build reproducible pierde valor para auditoría, bugfix y comparación futura.

4. Objetivos de producto y presupuestos maestros

4.1. Objetivo del vertical slice

StoreInitial representativa
1920 × 1080
calidad PC de referencia
hasta 8 clientes simultáneos
mobiliario y stock representativos
Golden Path y save/load
60 FPS objetivo
sin degradación acumulativa

Hasta seleccionar hardware mínimo, el PASS de H6 se refiere al **equipo de referencia registrado** y no autoriza requisitos de Steam.

4.2. Presupuesto maestro provisional

ID	Métrica	Target H6	Estado
PERF-TGT-001	Frame rate objetivo	60 FPS	heredado
PERF-TGT-002	Frame time nominal	16,67 ms	derivado
PERF-TGT-003	Resolución	1920 × 1080	heredado
PERF-TGT-004	Población	8 clientes simultáneos	heredado
PERF-TGT-005	Carga de StoreInitial	<5 s en equipo de referencia	heredado/provisional
PERF-TGT-006	Guardado/cierre	<2 s en equipo de referencia	heredado/provisional
PERF-TGT-007	Allocations en estado estable	próximas a cero	heredado
PERF-TGT-008	Excepciones recurrentes	0	vigente
PERF-TGT-009	Stutter persistente	0	vigente
PERF-TGT-010	Crecimiento de memoria en soak	no continuo	vigente
PERF-TGT-011	Bloqueo perceptible sin feedback	0	vigente
PERF-TGT-012	Regresión sin explicación	0	vigente

4.3. Frame pacing provisional para H6

En una captura válida de al menos diez minutos tras warm-up:

Métrica	PASS provisional	Observación
Mediana	≤16,67 ms	objetivo de 60 FPS
P95	≤20 ms	margen para variación normal
P99	≤33,33 ms	evita stutter frecuente
Frames >33,33 ms	<1 %	excluir transiciones registradas
Frames >50 ms	no repetitivos	cualquier patrón se investiga
Freeze >100 ms	0 durante gameplay estable	carga explícita se registra aparte

Estos límites son **provisionales** y deben ratificarse con datos del equipo de referencia. No sustituyen una revisión perceptual.

4.4. Presupuesto de CPU/GPU

Para mantener margen a 60 FPS:

Área	Envelope de planificación	Regla
Main Thread	≤12 ms estable	deja margen a render/driver/OS
Render Thread	sin cola sostenida bloqueante	observar waits
GPU	≤14 ms estable	no saturar 16,67 ms
Trabajo no crítico	distribuir o diferir	no concentrar en un frame
Pico de UI/acción	≤33,33 ms	sin repetición ni input perdido

No se aceptará “CPU 12 + GPU 14 = 26” como suma directa: CPU y GPU se solapan. El cuello se determina con Timeline, GPU Profiler y marcadores.

4.5. Presupuesto de memoria provisional

Sin hardware mínimo aprobado no se publica un máximo absoluto. Para H6 se aplican guardrails:

- memoria total estable tras warm-up;

- pico documentado al cargar StoreInitial;
- managed heap sin crecimiento continuo;
- no más de una tendencia sostenida tras repetir días, paneles y save/load;
- ninguna colección GC recurrente causada por un loop estable;
- snapshots antes/después de soak con retenciones explicadas;
- texturas, meshes, audio y render targets inventariados;
- buffers y pools con límites;
- cachés liberables al cambiar de escena.

Objetivo interno inicial para allocations:

Ruta	Target
Simulación estable	0 B/frame ideal; desviación justificada
Cámara/input continuo	0 B/frame
HUD sin cambio de datos	0 B/frame
Apertura de panel	allocation puntual permitida y medida
Save/load	allocation transitoria permitida, sin fuga
Spawning	allocation puntual; evitar repetición innecesaria

4.6. Presupuesto de carga y respuesta

Flujo	Target provisional	Feedback
Proceso a MainMenu usable	medir; objetivo inicial <5 s	logo/estado si supera percepción inme
MainMenu a StoreInitial	<5 s en referencia	loading state y bloqueo coherente
Carga de slot	<5 s en referencia	progreso/estado y error recuperable
Guardado manual/autosave	<2 s	indicador no intrusivo
Cierre de día + persistencia	<2 s para commit/guardado	no doble transición
Apertura de panel local	percepción inmediata; ideal <100 ms	estado visible
Confirmación de input	mismo frame o siguiente	feedback visual/audio

La carga de contenido y la restauración de save se medirán por separado para localizar coste.

4.7. Presupuesto de estabilidad

- siete jornadas sin reiniciar;
- al menos diez ciclos de MainMenu ↔ StoreInitial en prueba de lifecycle;
- save/load repetido;
- creación y retirada de mobiliario;
- apertura/cierre de paneles;
- cambio de calidad/resolución cuando exista;
- alt-tab y pérdida de foco;
- cero crash;
- cero error recurrente;
- cero duplicación de roots o audio loops;
- memoria sin pendiente ascendente no explicada;
- resultados deterministas dentro del contrato.

4.8. Presupuesto de build y almacenamiento

No se fija un tamaño final arbitrario antes del contenido final. Cada build registra:

- tamaño total;
- tamaño por escena/datos/managed/shaders cuando Build Report lo permita;

- variación contra baseline;
- assets dominantes;
- compresión;
- símbolos y debug separados;
- tamaño de save y backup;
- tiempo de instalación/arranque.

Una regresión de tamaño superior al 10 % entre builds comparables requiere explicación, aunque siga siendo pequeña.

4.9. Target de 30 FPS

30 FPS puede existir como cap opcional o fallback para hardware mínimo, pero:

- no reemplaza el target histórico de H6;
- no autoriza publicar requisitos;
- debe mantener frame pacing estable a 33,33 ms;
- la simulación no cambia resultados;
- cámara, input y UI se prueban a 30 y 60;
- cualquier animación o suavizado debe ser independiente del frame rate.

5. Hardware, sistema operativo y entornos

5.1. No publicar hardware antes de medir

Los campos de requisitos mínimos y recomendados de Steam permanecen TBD. No se copiarán especificaciones genéricas de otro juego ni se deducirán por el tamaño del proyecto.

5.2. Perfiles de hardware requeridos

Perfil	Finalidad	Estado
DEV-REF	equipo principal de desarrollo y H6	registrar
PC-MIN-CANDIDATE	candidato a requisitos mínimos	adquirir/seleccionar
PC-REC-CANDIDATE	candidato a recomendados 1080p60	seleccionar
PC-INTEGRATED	iGPU o GPU de entrada relevante	evaluar mercado
PC-LAPTOP	portátil con límites térmicos	evaluar sostenido
PC-HIGH	detectar caps, CPU bottleneck y escalabilidad	opcional
STEAM-DECK	compatibilidad/aspiración	no gate hasta decisión formal

5.3. Campos obligatorios del registro

- fabricante y modelo de CPU;
- núcleos/hilos;
- RAM y velocidad si es relevante;
- GPU y VRAM;
- driver;
- almacenamiento y espacio libre;
- OS, edición y build;
- resolución/refresh del monitor;
- modo de energía;
- temperatura/thermal throttling si aplica;
- procesos relevantes;
- versión Unity/build;

- locale;
- dispositivos de input;
- fecha.

5.4. Matriz de sistema operativo

El target inicial es Windows x64. Antes de release se probarán las versiones de Windows que se decidan soportar oficialmente. Cada entrada debe incluir:

- arranque;
- permisos de save;
- rutas de usuario;
- modo ventana/fullscreen;
- escalado DPI;
- audio;
- input;
- drivers;
- antivirus/SmartScreen cuando proceda;
- suspensión y reanudación si se declara soporte.

5.5. Almacenamiento

El objetivo histórico se midió “en equipo de desarrollo” sin especificar disco. La matriz debe incluir SSD y, si se pretende soportar HDD, una prueba real. No se publicará “SSD recomendado” o “requerido” sin comprobar impacto.

5.6. Temperatura y rendimiento sostenido

En portátil o hardware compacto se ejecutará un soak de al menos 30–60 minutos. Se registrará si el frame time empeora por temperatura. Un benchmark de dos minutos no demuestra estabilidad térmica.

5.7. Refresh rate y sincronización

Se probarán al menos 60 Hz y un refresh superior si está disponible. La política final debe contemplar:

- VSync on/off;
- caps 30/60/120/unlimited o conjunto aprobado;
- monitores VRR cuando sea posible;
- cambio de monitor;
- modo ventana y borderless;
- ausencia de tearing o consumo excesivo según opción.

6. Metodología de profiling

6.1. Build types

Build	Uso	Aceptación
Editor	diagnóstico rápido	no aprueba target
Development + Profiler	CPU/GPU/memoria detallada	diagnóstico y baseline técnica
Development sin Deep Profile	capturas comparables	recomendada para profiling regular
Deep Profile	localizar scripts puntuales	no comparar FPS global

Build	Uso	Aceptación
Release-like	rendimiento representativo	obligatorio antes de H6/RC
Release Candidate	aprobación final	obligatorio para requisitos publicados

6.2. Condiciones de una captura válida

1. build y hash identificados;
2. hardware y driver registrados;
3. misma resolución, calidad y modo de pantalla;
4. escena y save congelados;
5. población y contenido idénticos;
6. warm-up completado;
7. duración suficiente;
8. sin herramientas externas invasivas no registradas;
9. tres ejecuciones cuando la variación sea relevante;
10. resultado archivado con notas.

6.3. Warm-up

Antes de capturar gameplay estable:

- cargar la escena;
- permitir inicialización y compilación de shaders;
- abrir/cerrar una vez los paneles que puedan crear recursos;
- generar la población objetivo;
- esperar estabilización de memoria y audio;
- registrar cuánto duró el warm-up;
- comenzar la ventana de medida en un marker conocido.

Las cargas en frío se miden por separado y no se “calientan”.

6.4. Repeticiones

Para benchmarks cortos se ejecutan tres pasadas. Se informa mediana y rango. Si una pasada difiere materialmente, no se descarta sin investigar.

6.5. Herramientas Unity

- Profiler Timeline;
- CPU Usage;
- GPU Usage cuando el backend lo soporte;
- Rendering;
- Memory;
- Audio;
- UI Details/UI Profiler cuando esté disponible;
- Frame Debugger;
- Memory Profiler package cuando se apruebe su incorporación;
- Build Report;
- Profile Analyzer cuando se necesiten comparaciones agregadas.

La incorporación de paquetes de profiling se registra y no invade runtime de release.

6.6. Métricas mínimas por captura

- frame time total;
- main thread;
- render thread;

- GPU;
- batches y SetPass;
- triangles/vertices visibles cuando sea útil;
- shadow casters;
- UI rebuild/layout;
- GC Alloc;
- managed heap;
- total used memory;
- textures/meshes/audio;
- tiempos de carga/save;
- markers de sistema;
- logs y excepciones.

6.7. Determinación de cuello de botella

- **CPU bound:** main/render thread limita y GPU queda por debajo;
- **GPU bound:** GPU supera el presupuesto mientras CPU espera;
- **I/O bound:** lectura/escritura o descompresión domina transiciones;
- **memory bound:** presión, paging, picos o GC generan stutter;
- **synchronization bound:** waits, locks o fence dominan;
- **content bound:** densidad, materiales, luces o UI elevan coste;
- **configuration bound:** resolución, VSync, quality o Development flags alteran resultado.

No se optimiza GPU cuando el cuello es CPU ni se reduce arte para ocultar una espera de save.

6.8. Marcadores recomendados

Convención:

CC.<System>.<Operation>

Ejemplos:

- CC.Scene.LoadSceneInitial;
- CC.Persistence.SerializeSlot;
- CC.Persistence.WriteAtomic;
- CC.Customers.Tick;
- CC.Customers.PathRequest;
- CC.Inventory.VisualSync;
- CC.UI.StoreHudRefresh;
- CC.Placement.ValidatePreview;
- CC.DayCycle.CloseDay;
- CC.Audio.RouteEvent.

Los markers no deben crear strings por frame ni permanecer en exceso en Release si su coste es material.

6.9. Baseline storage

Estructura recomendada:

```
Documentation/10_Development_Records/Performance/  
  <BuildID>/  
    Performance_Report.md  
    Hardware_Record.md  
    Scenario_Data.csv  
    Captures/  
    Memory/  
    Logs/  
    Screenshots/
```

El manifest 32 referenciará los artefactos de cierre, no cada captura temporal.

7. Catálogo de escenarios de rendimiento

7.1. PERF-SCN-001 — Cold Boot

Desde proceso no ejecutado hasta MainMenu usable.

Registrar:

- tiempo total;
- carga de assemblies y primera escena;
- creación de ApplicationRoot;
- transición a MainMenu;
- primer input aceptado;
- memoria inicial;
- log.

7.2. PERF-SCN-002 — MainMenu estable

MainMenu durante cinco minutos, incluyendo:

- navegación por slots;
- opciones;
- ES/EN;
- UI/texto 150 %;
- ratón y teclado;
- cambio de resolución futuro.

Objetivo: sin allocations recurrentes, foco estable y frame pacing consistente.

7.3. PERF-SCN-003 — Nueva partida

Crear slot, inicializar estado, cargar StoreInitial y alcanzar control jugable. Separar:

- creación de snapshot;
- transición de escena;
- composición runtime;
- registro de catálogos;
- visual sync;
- audio;
- primer frame usable.

7.4. PERF-SCN-004 — StoreInitial vacía

Escena representativa sin clientes, con mobiliario y stock base. Sirve como baseline de render y servicios.

7.5. PERF-SCN-005 — StoreInitial equipada

Mobiliario y productos representativos completos. Medir cámara en posiciones que maximizan visibilidad, transparencias, sombras y UI.

7.6. PERF-SCN-006 — Ocho clientes

Hasta ocho clientes simultáneos recorriendo browsing, selección, reserva, cola, checkout y salida. Incluir rutas fallidas controladas y puerta automática.

7.7. PERF-SCN-007 — Pico de checkout

Cola máxima representativa, ventas consecutivas, feedback, inventario, ledger, audio y resultados parciales. Busca spikes por transacción y UI.

7.8. PERF-SCN-008 — Build Mode intensivo

Entrar/salir, mover preview, rotar, validar celdas, colocar, mover y retirar múltiples objetos. Busca:

- validación por frame;
- raycasts;
- material/ghost;
- occupancy;
- visual sync;
- GC;
- click-through de UI.

7.9. PERF-SCN-009 — Operations/UI pesada

Abrir todas las pestañas, listas, modales, tutorial, resultados y errores en ES/EN y escala extrema. Observar Canvas rebuild, layouts, strings y event subscriptions.

7.10. PERF-SCN-010 — Pedido y recepción

Crear pedido, entregar, representar caja, transferir a almacén, asignar display y reponer. Busca instanciación y sincronización de stock.

7.11. PERF-SCN-011 — Cierre de día

Warning, resolución de clientes, transacciones pendientes, resultados, autosave y transición. Medir cada fase con markers.

7.12. PERF-SCN-012 — Save manual/autosave

Capturar serialización, checksum, escritura temporal, flush, verificación, reemplazo y backup. Probar save pequeño y estado representativo.

7.13. PERF-SCN-013 — Load de slot

Cargar primario, validar, migrar si procede, reconstruir servicios, restaurar visuales y aceptar input. Probar primario válido y recuperación de backup por separado.

7.14. PERF-SCN-014 — Siete días

Golden Path variado durante siete jornadas sin reiniciar. Registrar memoria, frame time, objetos, logs, save size y tiempos por día.

7.15. PERF-SCN-015 — Scene lifecycle

Diez ciclos MainMenu ↔ StoreInitial. Verificar roots, listeners, loops de audio, caches, memoria y duplicaciones.

7.16. PERF-SCN-016 — Stress progresivo

Superar gradualmente ocho clientes y densidad esperada para encontrar el punto de degradación. No es criterio de producto; sirve para margen y comportamiento seguro.

7.17. PERF-SCN-017 — Accesibilidad extrema

UI 150 %, texto 150 %, reducción de movimiento, idioma expansivo, resolución mínima, modo ventana y opciones de audio. Debe seguir dentro de límites aceptables.

7.18. PERF-SCN-018 — Calidad gráfica

Ejecutar cada preset aprobado en un mismo hardware y save. Verificar que la diferencia de coste corresponde a la diferencia visual y que no rompa legibilidad.

7.19. PERF-SCN-019 — Pérdida de foco

Alt-tab, minimizar, recuperar foco, cambiar dispositivo y volver. Observar audio, input, tiempo, CPU en background y frame pacing.

7.20. PERF-SCN-020 — Sesión térmica

60 minutos en portátil/candidato mínimo con carga representativa. Registrar clocks/temperatura si es posible y degradación de percentiles.

8. CPU, simulación y scheduling

8.1. Rutas críticas

- input continuo;
- cámara;
- placement preview;
- wall occlusion;
- customer tick;
- spawner;
- navegación;
- shopping/reservations;
- cola/checkout;
- ciclo diario;
- HUD;
- sincronización visual;
- audio routing;
- guardado/carga.

8.2. Update por entidad

El proyecto actual usa Update por agente de cliente y por algunos controladores. Para ocho clientes puede ser suficiente. Antes de refactorizar se medirá:

- coste por agente;
- coste total a 8, 16 y stress;
- frecuencia necesaria por estado;
- allocations;
- pathfinding;
- animación/presentación.

Si el coste escala de forma problemática, se introducirá un scheduler central o ticks escalonados conservando tiempos y determinismo.

8.3. Frecuencias recomendadas

Trabajo	Frecuencia orientativa
Input/cámara	por frame
Movimiento visible	por frame/Unity lifecycle
HUD sin cambios	event-driven; refresh limitado
Decisión de compra	al evaluar o por tick bajo
Paciencia	tick estable, no necesariamente por frame
Spawning	acumulador/intervalo
Día lógico	tick configurable
Stock visual	al cambiar estado o batch
Save	eventos seguros
Logs/diagnóstico	agregados/rate-limited

8.4. ResolveReferences

Las búsquedas de referencias dentro de Update deben auditarse. Alternativas, por orden:

1. referencia serializada;
2. composición explícita;
3. registro de servicios;
4. cache tras Awake/Start;
5. resolución eventual limitada y desactivada tras éxito;
6. búsqueda global solo como fallback diagnosticado.

No se reemplazará por un singleton global sin analizar lifecycle y tests.

8.5. Búsquedas y colecciones

- evitar Find* en loops calientes;
- cachear GetComponent local;
- no recrear listas por frame;
- usar diccionarios/índices cuando la consulta lo justifique;
- no convertir toda colección a array/list por comodidad;
- ordenar solo cuando cambia la fuente;
- no usar nombres de GameObject como contrato;
- medir antes de sustituir una lista pequeña por estructura compleja.

8.6. Strings y logs

- el HUD formatea al cambiar datos o a frecuencia limitada;
- logs de éxito no se emiten cada tick;
- errores repetidos se agregan;
- IDs y valores se almacenan sin interpolación innecesaria;
- builds de release reducen diagnóstico no esencial;
- ningún log sensible o enorme bloquea frame.

8.7. Physics y raycasts

El placement y puertas deben:

- usar layer masks;
- evitar raycasts redundantes;
- reutilizar buffers NonAlloc cuando exista volumen medido;
- no consultar toda la escena;

- separar collider visual y técnico;
- no aumentar fixed timestep para “arreglar” movimiento sin medir.

8.8. Jobs/Burst/DOTS

No se introducirán como respuesta automática. Solo se evalúan si:

- un trabajo CPU dominante es paralelizable;
- la arquitectura actual no cumple target;
- el coste de integración y determinismo es justificable;
- existe benchmark del prototipo;
- no invade Domain o authoring innecesariamente.

Para ocho clientes, claridad y seguridad probablemente tienen mayor valor que una migración tecnológica.

9. Memoria, GC y lifecycle

9.1. Tipos de memoria a observar

- managed heap;
- native memory;
- graphics/VRAM estimada;
- textures;
- meshes;
- audio clips;
- render textures;
- scene objects;
- animation;
- navigation;
- serialized data;
- caches y pools.

9.2. Baseline de memoria

Capturas mínimas:

1. MainMenu tras warm-up;
2. StoreInitial recién cargada;
3. equipada sin clientes;
4. ocho clientes;
5. cierre de día;
6. después de save/load;
7. tras siete días;
8. tras volver a MainMenu;
9. tras diez ciclos de escena.

9.3. Leak vs cache

Una retención no es automáticamente leak. Debe clasificarse:

- cache intencional con límite;
- servicio persistente;
- asset cargado por escena;
- referencia estática residual;
- suscripción no liberada;

- objeto duplicado;
- pool sin límite;
- asset que Unity conserva por política.

Cada cache persistente debe documentar owner, máximo, invalidación y liberación.

9.4. Allocations por frame

Se priorizan:

- cámara/input;
- customer tick;
- HUD;
- placement;
- wall occlusion;
- audio routing.

Una allocation pequeña puede acumular GC. Se mide en una ventana estable y se identifica el call stack. No se sustituyen strings o colecciones por código ilegible si el coste es irrelevante.

9.5. GC incremental

Está activado. Se comprobará:

- frecuencia de collections;
- slices y spikes;
- interacción con allocations persistentes;
- coste durante UI, save y spawning;
- resultado en Release-like.

No se considera solución a generar basura continuamente.

9.6. Pooling

Pooling se usa cuando:

- existe churn medido;
- la creación/destrucción causa spikes;
- el objeto tiene reset seguro;
- el máximo está acotado;
- no conserva suscripciones o estado;
- reduce coste total y no solo GC.

Candidatos condicionales:

- VFX;
- feedback flotante;
- clientes si su lifecycle lo justifica;
- producto visual unitario;
- filas/list items si la lista crece;
- AudioSources adicionales.

9.7. Materiales y memoria

Evitar acceso repetido a `renderer.material` que cree instancias. Preferir:

- shared materials;
- MaterialPropertyBlock cuando proceda;
- paletas/variantes;

- atlas/trims;
- instancias explícitas con lifecycle.

9.8. Save size

Cada reporte de persistencia registra:

- bytes de primario;
- bytes de backup;
- tiempo de serialización;
- tiempo de escritura;
- tiempo de verificación;
- número de entidades;
- versión de schema;
- compresión si se introduce.

No se comprimirá un save pequeño si aumenta latencia y complejidad sin beneficio.

10. Renderizado y URP

10.1. Orden de investigación GPU

1. confirmar GPU bound;
2. resolución/render scale;
3. sombras;
4. luces adicionales;
5. SSAO;
6. opaque/depth textures;
7. transparencias/overdraw;
8. materiales/SetPass;
9. postprocesos;
10. geometría/LOD/culling;
11. shader variants y hitches.

No se reduce geometría primero si el coste dominante son sombras o fill rate.

10.2. Resolución y render scale

El target histórico es 1080p. Se probarán:

- 1280×720;
- 1600×900 si se soporta;
- 1920×1080;
- 2560×1440;
- resolución mínima de UI;
- render scale 1.0 y valores menores solo como opción explícita.

La escala dinámica no se introduce sin un diseño de calidad y pruebas de legibilidad.

10.3. Sombras

La configuración observada usa sombras de luz principal y adicionales, soft shadows, alta resolución y distancia amplia para una tienda pequeña. Se medirá:

- coste de shadow map;
- número de casters;
- cascadas necesarias;

- distancia real de cámara;
- additional light shadows;
- calidad visual Low/Medium/High;
- artefactos y popping.

Posibles niveles:

- Low: sombras principales reducidas, sin additional shadows;
- Medium: principal + distancia moderada;
- High: configuración representativa aprobada.

Los valores exactos se fijan tras profiling.

10.4. Luces

- limitar luces adicionales visibles por objeto;
- evitar solapamiento innecesario;
- usar baked/mixed cuando preserve cambios de día y authoring;
- emisión no sustituye iluminación;
- revisar luces de señalética;
- no añadir una luz por producto;
- usar light probes si aportan a personajes.

10.5. SSAO

SSAO está activo. Se probará:

- calidad visual con/sin;
- downsample;
- sample count;
- coste a 720p/1080p/1440p;
- interacción con estilización;
- opción de calidad si es significativa.

No se mantendrá por inercia si el color blocking, sombras y materiales ya aportan separación suficiente.

10.6. Depth y Opaque Texture

Ambas están requeridas. Se inventariarán features que realmente las consumen. Si ninguna feature aprobada necesita opaque texture, su desactivación puede ahorrar coste; si se usa para vidrio, outline o VFX, se mantiene y documenta.

10.7. Transparencias

Riesgos:

- vidrio de fachada;
- UI world-space futura;
- partículas;
- hair/cards futuros;
- decals/transparencias de producto.

Se revisa overdraw con Frame Debugger y escena densa. El vidrio debe ser legible sin capas redundantes.

10.8. SRP Batcher y materiales

SRP Batcher está activo. Para beneficiarse:

- shaders compatibles;
- materiales compartidos;
- evitar variantes arbitrarias;
- no instanciar material por objeto;
- registrar custom shaders;
- comprobar SetPass/batches en Player.

10.9. Dynamic batching y GPU Resident Drawer

Dynamic batching y GPU Resident Drawer están desactivados. No se activarán por “mejorar performance” sin un A/B test porque pueden no beneficiar esta escena, aumentar variantes o introducir incompatibilidades.

10.10. Postproceso

El perfil contiene componentes, varios con intensidad cero. Se revisará:

- coste de componentes activos aunque estén en cero;
- tonemapping;
- bloom;
- vignette;
- motion blur;
- depth of field;
- chromatic aberration;
- film grain.

Motion blur, DOF, chromatic aberration y grain no son requisitos de identidad y deben permanecer desactivables; reducción de movimiento prevalece.

10.11. Shader compilation y variants

Antes de RC:

- Build Report de shaders;
- variants por URP feature;
- stripping validado;
- hitches de primera aparición;
- warm-up solo para variants realmente usadas;
- no incluir features XR/mobile/tardías sin necesidad;
- comparar build size y tiempos.

11. Arte, geometría, materiales y contenido visual

11.1. Presupuestos heredados de Art Bible

Categoría	LOD0 orientativo	Textura orientativa
Producto pequeño	500–3.000 tris	256–1024
Packaging	100–1.500 tris	256–1024
Mueble	2.000–15.000 tris	512–2048
Arquitectura modular	500–10.000 tris/módulo	tiling/trim

Categoría	LOD0 orientativo	Textura orientativa
Prop decorativo	200-5.000 tris	256-1024
Personaje estilizado	15.000-35.000 tris	1024-2048

Son guardrails, no aceptación. Materiales, sombras, overdraw y scripts pueden dominar más que los tris.

11.2. LOD

El generador histórico usó ratios aproximados 60 %/25 %. La validación se hará en cámara real:

- sin popping severo;
- silueta y rol conservados;
- producto pequeño agrupado cuando corresponda;
- distancias coherentes entre familias;
- no crear LOD que aumenta batches sin ahorro;
- crossfade solo si el coste/arte lo justifica;
- personajes validados en grupo.

11.3. Culling

- frustum culling se asume, pero se comprueba bounds;
- occlusion culling se evalúa solo si la geometría de tienda lo beneficia;
- wall occlusion de gameplay no equivale a occlusion culling de render;
- objetos ocultos por paneles no deben seguir actualizando visuales costosos;
- arquitectura fuera de cámara no debe tener bounds gigantes.

11.4. Productos visibles

La capacidad lógica no obliga a instanciar cada unidad. Estrategias:

- estados de llenado;
- grupos de cajas;
- variantes por nivel de stock;
- mallas combinadas controladas;
- slots limitados;
- pooling si existe churn.

La representación debe seguir permitiendo leer categoría, disponibilidad y cambio de stock.

11.5. Texturas

Actualmente no hay texturas de producción. Antes de integrarlas:

- tamaños por categoría;
- mipmaps;
- compresión por PC;
- atlas/trims;
- lectura a cámara;
- evitar microtexto y moiré;
- streaming mipmaps solo tras medir memoria/carga;
- no usar 2K por defecto en props pequeños;
- conservar fuentes y licencias fuera de build cuando proceda.

11.6. Colliders

Colliders simples y compuestos suelen ser preferibles a MeshCollider complejo. Se revisa:

- número por objeto;
- contacto real necesario;
- layers;
- triggers;
- coste de puerta;
- coherencia con NavMesh;
- no colisionar cada producto si basta el mueble.

11.7. Animación

Con personajes finales:

- Animator count;
- culling mode;
- rig complexity;
- skin weights;
- update offscreen;
- root motion o movimiento autoritativo;
- clip sampling;
- transitions;
- coste a ocho y stress.

No se aprobará un personaje solo en aislamiento.

11.8. VFX

VFX final aún no existe. Reglas:

- feedback breve;
- límites de partículas;
- pooling condicional;
- reducción de movimiento;
- no usar transparencia fullscreen;
- calidad escalable;
- no emitir por frame sin cambio;
- profiler con acciones repetidas.

12. UI, texto y localización

12.1. UI como ruta crítica

El proyecto usa uGUI/TextMeshPro. Riesgos:

- rebuild de Canvas;
- layout groups profundos;
- ContentSizeFitter recursivo;
- listas recreadas;
- strings por frame;
- animaciones de panel;
- raycast targets innecesarios;
- fuentes/atlases;
- pseudolocalización;
- escalado 150 %.

12.2. Canvas

- separar zonas que cambian frecuentemente de fondos estáticos;
- no dividir en docenas de Canvas sin medir;
- desactivar paneles no visibles;
- limitar GraphicRaycaster a lo necesario;
- no mover grandes jerarquías cada frame;
- comprobar rebuild al actualizar dinero, hora e inventario.

12.3. HUD

StoreHudScreen refresca como máximo cada 0,25 s. Esta limitación es razonable, pero debe medirse:

- coste de cada refresh;
- strings;
- layout;
- acceso a snapshot;
- ausencia de refresh cuando no hay root;
- actualización inmediata de eventos críticos.

Se puede evolucionar a event-driven sin perder un refresh de seguridad si el coste lo justifica.

12.4. Listas

Listas de slots, productos, pedidos o resultados:

- reutilizar filas cuando crezcan;
- diferenciar datos y vista;
- no destruir/recrear todo ante un cambio pequeño;
- ordenar al cambiar fuente;
- virtualizar solo cuando el volumen real lo exija;
- mantener foco y navegación.

12.5. Texto y fuentes

Se probarán:

- ES;
- EN;
- pseudolocale +30 %;
- UI/texto 80-150 %;
- resolución mínima;
- fallback de glyphs;
- atlas dinámico/estático según decisión;
- memoria y generación de glyphs;
- ausencia de hitch al mostrar texto nuevo.

12.6. Estados de carga

Un loading spinner no debe ejecutar layout complejo por frame. Las animaciones usan un coste acotado y respetan reducción de movimiento. Si no existe progreso real, se comunica estado sin inventar porcentaje.

12.7. Input y latencia UI

- foco visible el mismo frame;
- click no atraviesa al mundo;

- modal bloquea input correctamente;
 - navegación no depende de FPS;
 - repetición de tecla/control acotada;
 - no registrar callbacks duplicados al abrir paneles;
 - cleanup al cerrar/cambiar escena.
-

13. Clientes, navegación e IA

13.1. Objetivo de población

Ocho clientes simultáneos es el escenario representativo. Debe incluir estados diferentes, no ocho agentes idle.

13.2. Costes a separar

- lógica de perfil/interés;
- reserva de producto;
- búsqueda de display;
- path request;
- movimiento NavMesh;
- paciencia;
- dwell;
- cola;
- checkout;
- animación;
- visuales/audio;
- destrucción/salida.

13.3. Pathfinding

- no pedir ruta cada frame;
- reutilizar destino mientras sea válido;
- backoff ante fallo;
- liberar reserva si queda bloqueado;
- limitar reintentos;
- no rebakear NavMesh por preview;
- medir spikes de path en apertura o cambios de layout;
- validar cambios de mobiliario y acceso.

13.4. Spawning

El spawner actual usa acumulador y Update. Se probará:

- coste de selección de perfil;
- determinismo;
- cola de solicitudes;
- instanciación;
- registro visual;
- warm-up/pool si procede;
- límite duro;
- ausencia de burst accidental tras pausa.

13.5. Tick escalonado

Si la CPU lo exige, clientes pueden distribuir decisiones en varios frames. Requisitos:

- resultado determinista o tolerancia definida;
- no retrasar feedback crítico;
- no crear hambre de actualización;
- tiempo lógico correcto;
- pruebas a 30/60 FPS;
- configuración central.

13.6. Stress y degradación segura

Al superar población objetivo:

- no crash;
 - no duplicar clientes;
 - no perder reservas;
 - no bloquear cierre;
 - se puede reducir frecuencia de decisiones no críticas;
 - animaciones offscreen pueden simplificarse;
 - el sistema debe registrar límite, no ocultarlo.
-

14. Carga de escenas, composición y lifecycle

14.1. Scene flow

El proyecto documenta transiciones asíncronas y rechazo de transiciones concurrentes. Se medirán:

- solicitud;
- fade/loading state;
- operación async;
- activación;
- composición;
- primer frame usable;
- liberación de escena anterior.

14.2. StoreInitial

Antes de profiling válido:

1. debe integrar assets representativos;
2. debe usar catálogos y colliders correctos;
3. debe desactivar el shell procedural en la ruta aprobada;
4. debe entrar en un Build Profile de profiling;
5. debe pasar Golden Path;
6. debe tener save/load;
7. debe evitar duplicación con Store.

14.3. Runtime builders

Los builders representativos y blockout pueden ser útiles para tests/fallback. En ruta final:

- no regenerar arquitectura fija en cada carga si puede authorarse;
- medir instanciación de stock/mobiliario;
- separar authoring de runtime;
- no usar búsquedas por nombre como composición principal;
- conservar fallback aislado y diagnosticable.

14.4. Catálogos

Los catálogos deben cargar una vez por lifecycle apropiado. Se prohíbe:

- escanear todos los assets por frame;
- duplicar registros;
- reconstruir listas estables en cada panel;
- mantener referencias a escenas descargadas.

14.5. Resources y carga directa

Se observan fallbacks con `Resources.Load/FindObjectsOfTypeAll`. Mientras el contenido sea pequeño puede ser aceptable, pero:

- toda ruta debe ser explícita;
- se mide coste;
- no se usa como service locator;
- no se introduce Addressables sin una necesidad real de streaming, DLC o tamaño;
- si se migra, se crea plan de carga, error y build.

14.6. Duplicación

Tests y profiling deben detectar:

- ApplicationRoot múltiple;
- audio loops duplicados;
- input maps suscritos varias veces;
- EventSystem/cámara duplicados;
- catálogos repetidos;
- static state residual;
- GameObjects DontDestroyOnLoad huérfanos.

15. Persistencia, guardado y carga

15.1. Rendimiento no compromete integridad

No se omite checksum, verificación, backup o atomicidad para reducir milisegundos sin una decisión formal. La integridad tiene prioridad sobre el target de 2 s; si el target falla, se optimiza implementación o feedback, no se escribe de forma insegura.

15.2. Fases de save

Markers separados:

1. capturar snapshot;
2. normalizar/validar;
3. serializar;
4. calcular checksum;
5. escribir .tmp;
6. flush;
7. releer/verificar;
8. rotar backup;
9. reemplazar primario;
10. notificar resultado.

15.3. Fases de load

1. localizar archivos;
2. leer primario;
3. validar envelope/checksum;
4. deserializar;
5. migrar;
6. reparar permitido;
7. restaurar agregados;
8. reconstruir vistas;
9. verificar equivalencia;
10. aceptar input.

15.4. Hilos y asincronía

La asincronía puede mejorar respuesta, pero:

- Unity objects se manipulan en main thread;
- no se introduce race en snapshot;
- se conserva orden transaccional;
- cancellation y cierre son seguros;
- errores vuelven al main thread con feedback;
- tests cubren interrupción.

Antes de mover serialización/I/O a background se mide cuánto tiempo bloquea realmente.

15.5. Autosave

- no disparar múltiples autosaves concurrentes;
- coalescer solicitudes;
- no guardar cada cambio pequeño;
- mostrar estado;
- no cerrar aplicación hasta resultado o política segura;
- mantener último save válido.

15.6. Crecimiento del save

Cada nuevo sistema estima:

- registros por entidad;
- días/historial retenido;
- logs persistentes;
- IDs y strings repetidos;
- compatibilidad de schema;
- impacto de backup.

No se persiste cache derivable sin necesidad.

16. Audio y rendimiento

16.1. Estado observado

- 10 WAV;
- cuatro canales runtime;
- una fuente compartida por canal como base;
- 32 voces reales/512 virtuales en ProjectSettings, no presupuesto de diseño;
- clips cortos preloaded;

- todo audio actual 2D;
- sin AudioMixer final;
- sin streaming final.

16.2. Presupuesto de voces

Objetivo de diseño heredado del Audio Plan:

Familia	Voces orientativas
Music	1
Ambience	1
UI	4-6
Effects	12-20 tras profiling
Characters	agregadas/priorizadas por distancia

No se eleva el límite global para ocultar falta de concurrency policy.

16.3. Load type

- UI/one-shots cortos: Decompress On Load cuando la latencia importe;
- SFX medianos: Compressed In Memory si reduce memoria sin latencia problemática;
- música/ambiente largos: Streaming tras medir;
- mono para SFX espacializados;
- estéreo solo si aporta.

16.4. Spam y cooldown

Eventos repetidos pueden costar CPU, voces y fatiga. Se aplican:

- cooldown;
- concurrency groups;
- prioridad;
- agregación de ventas/feedback;
- debounce de hover;
- state edge para puerta;
- rate limit de warnings.

16.5. Profiling

Medir:

- voces reales/virtuales;
 - CPU audio;
 - memoria de clips;
 - streaming;
 - latencia;
 - duplicación tras escenas;
 - loops;
 - cambio de volumen;
 - tienda con ocho clientes.
-

17. Input, cámara, oclusión y accesibilidad

17.1. Independencia del frame rate

Movimiento, rotación, zoom, temporizadores y repetición de input deben probarse a 30, 60 y FPS no limitados. No se multiplicará por deltaTime dos veces ni se asumirán 60 frames.

17.2. Cámara

OrbitCameraRig aplica pose en LateUpdate. Medir:

- coste;
- jitter;
- raycasts si se añaden;
- wall occlusion;
- interpolación;
- límites;
- reducción de movimiento;
- respuesta a frame pacing irregular.

17.3. Wall occlusion

PhaseWallOcclusionController opera en LateUpdate. Debe medirse con:

- número de paredes;
- frecuencia de raycasts/queries;
- restauración de materiales/renderers;
- cambio de objetivo;
- opción activada/desactivada;
- cámara rápida;
- materiales compartidos;
- ausencia de allocations.

17.4. Reducción de movimiento

No debe aumentar trabajo por frame. Cuando se activa puede:

- eliminar animaciones decorativas;
- reducir transiciones;
- desactivar motion blur;
- limitar camera shake;
- simplificar VFX.

Se valida que el cambio no deje coroutines/animators activos invisibles.

17.5. Escalado de UI/texto

UI y texto a 150 % pueden aumentar layout, geometría de texto y draw calls. Se perfilan como requisito, no se excluyen del target por ser “opción extrema”.

17.6. Opciones de rendimiento accesibles

- etiquetas claras;
- presets comprensibles;
- reversión de resolución;
- preview segura;
- no ocultar efectos funcionales;
- defaults razonables;

- reset;
- navegación completa por input soportado.

18. Quality Settings, frame cap y opciones gráficas

18.1. Problema actual

Solo existen perfiles Mobile y PC, VSync está a cero y no se localizó cap explícito. Para PC/Steam se necesita una política de usuario y benchmarks reproducibles.

18.2. Presets propuestos

Preset	Objetivo	Estado
Low	hardware mínimo candidato / claridad intacta	diseñar
Medium	equilibrio	diseñar
High	target visual 1080p60 recomendado	diseñar
Custom	combinación controlada	opcional

No es necesario crear Ultra si no existe diferencia visual significativa.

18.3. Opciones candidatas

- display mode;
- resolución;
- VSync;
- frame cap;
- preset;
- render scale;
- shadow quality/distance;
- SSAO;
- postprocessing;
- anti-aliasing;
- texture quality;
- anisotropic filtering;
- VFX density;
- reduced motion coordinada con accesibilidad.

Cada opción debe tener efecto medible, persistir, revertirse y no romper UI.

18.4. VSync y frame cap

Política propuesta para validar:

- default: VSync o cap 60 según comportamiento/tearing;
- opciones: 30, 60, 120 y Unlimited si el soporte es estable;
- no cap + VSync off en benchmarks salvo escenario explícito;
- registrar refresh rate;
- usar una sola fuente de control;
- comprobar que el cap no introduce pacing errático.

La decisión final requiere A/B test en monitores y hardware distintos.

18.5. Resolución y ventana

La configuración 1024×768 y no redimensionable debe sustituirse o justificarse antes de publicación. Se requiere:

- borderless/windowed/fullscreen aprobado;
- redimensionado seguro;
- DPI scaling;
- confirmación temporal y rollback;
- persistencia global;
- fallback si una resolución deja la UI inutilizable.

18.6. Auto-detección

No se implementará un autodetector complejo sin necesidad. Puede elegirse preset inicial conservador y permitir ajuste. Si se detecta hardware, la regla debe ser transparente, reversible y testeada.

19. Build, compilación y configuración de release

19.1. Perfiles necesarios

1. Windows_Development para desarrollo funcional.
2. Windows_Profiling con StoreInitial, Development, profiler permitido y símbolos controlados.
3. Windows_ReleaseCandidate sin TestLab ni tooling técnico, configuración release-like.
4. Windows_Release derivado de RC aprobada.

Los nombres son propuesta; el Build Guide mantiene autoridad sobre versionado.

19.2. StoreInitial y TestLab

- StoreInitial entra en profiling/RC solo después del gate de integración.
- TestLab no entra en RC/release.
- el preflight falla si escenas incorrectas están habilitadas.
- la lista se registra en manifest.

19.3. Development overhead

Se comparará Development vs Release-like para comprender:

- checks;
- logging;
- profiler support;
- script debugging;
- stripping;
- burst/jobs si se incorporan;
- build compression.

No se usa Deep Profile para afirmar FPS final.

19.4. Logs

- Player.log activo para diagnóstico;
- cero spam recurrente;
- niveles de logging por build;
- rotación/tamaño razonable;

- no registrar cada frame;
- errores de catálogo, save y escena suficientemente contextuales.

19.5. Stripping

stripEngineCode está activado. Cualquier reflexión, serialización o carga indirecta debe probarse en Release-like. Un build más pequeño que rompe contenido no es optimización.

19.6. Compresión de build

Se registra impacto en:

- tamaño;
- tiempo de build;
- instalación;
- primera carga;
- patching de Steam futuro.

La selección se alinea con Build Guide y Steamworks.

19.7. Reproducibilidad

Todo benchmark de gate conserva:

- commit/SHA;
 - versión Unity;
 - package manifest/lock;
 - Build Profile;
 - scripting defines;
 - calidad;
 - scenes;
 - build ID;
 - checksum;
 - logs.
-

20. Escalabilidad y sistemas futuros

20.1. Regla de reapertura de presupuestos

Cada sistema post-H6 reabre el presupuesto:

- empleados;
- puestos informáticos;
- investigación visual;
- ecommerce/logística;
- publishing/desarrollo interno;
- plataforma;
- infraestructura;
- múltiples tiendas.

No se asume que la baseline de ocho clientes cubre etapas tardías.

20.2. Empleados

Añadirán:

- agentes;

- navegación;
- colas de tareas;
- reservas;
- animación;
- UI;
- persistencia.

Antes de producción se crea un escenario de carga y presupuesto por empleado.

20.3. Expansión física

Más superficie, cámaras, objetos y rutas exigirán:

- culling;
- sectores o activación;
- NavMesh;
- LOD;
- luces;
- save size;
- densidad de productos.

No se optimiza ahora para el mapa final, pero se evita acoplar sistemas a una sola sala.

20.4. Servicios online del juego

Los sistemas ficticios de plataforma/infraestructura son gameplay local salvo decisión futura. No requieren networking real para representarse. No se introducirá backend real como “preparación”.

20.5. Steam Deck

Es una posible oportunidad, no compromiso. Requiere:

- resolución y UI;
- input completo;
- 30/40/60 FPS según objetivo;
- consumo y térmicas;
- suspend/resume;
- compatibilidad Proton;
- texto;
- almacenamiento.

Solo entra en requisitos tras decisión formal y pruebas físicas.

21. Flujo de optimización y control de cambios

21.1. Intake

Un issue de rendimiento debe incluir:

- síntoma;
- escenario;
- severidad;
- build;
- hardware;
- pasos;
- métrica;
- evidencia;

- regresión conocida;
- impacto de usuario.

21.2. Clasificación

Clase	Ejemplos
CPU	scripts, UI, navegación, serialización
GPU	sombras, SSAO, overdraw, materiales
Memory	leak, asset, cache, pool
GC	strings, listas, closures
I/O	scene, save, audio streaming
Pacing	VSync, cap, spikes
Lifecycle	duplicados, suscripciones, unload
Build	Development overhead, stripping
Perceived	feedback, input lag, loading state

21.3. Hipótesis

La hipótesis debe poder refutarse. Ejemplo:

“El spike de 45 ms al abrir Operations procede de reconstruir todas las filas y layouts, no de carga de datos.”

Se añade marker o prueba que permita confirmarlo.

21.4. Cambio mínimo

Se prioriza:

1. eliminar trabajo innecesario;
2. reducir frecuencia;
3. cachear con lifecycle;
4. batch;
5. reutilizar;
6. cambiar algoritmo/estructura;
7. reducir calidad escalable;
8. paralelizar;
9. migrar tecnología.

21.5. Verificación

- misma build base y escenario;
- tres pasadas si procede;
- antes/después;
- funcional y visual;
- accesibilidad;
- memoria;
- tests;
- no empeorar otro hardware;
- registrar side effects.

21.6. Reversión

Si la mejora no es reproducible o rompe claridad, se revierte. El tiempo invertido no justifica mantener complejidad.

21.7. Deuda aceptada

Requiere:

- límite cuantificado;
- escenario afectado;
- razón;
- owner;
- trigger;
- fecha/hito de revisión;
- mitigación para usuario;
- ausencia de S0/S1.

22. QA de rendimiento y regresión

22.1. Severidades

Severidad	Criterio de rendimiento
S0	crash, corrupción, bloqueo total o imposibilidad de iniciar por recursos
S1	Golden Path no usable, freeze repetido, fuga crítica, save/load bloqueante, target incumplido gr
S2	stutter notable, regresión significativa, opción gráfica defectuosa, carga excesiva con workarou
S3	degradación menor, coste localizado, issue cosmético/perceptual no bloqueante
S4	mejora u observación

22.2. Regresión significativa

Se investiga automáticamente cuando, en escenario comparable:

- frame time mediano empeora >10 %;
- P95/P99 empeora >15 %;
- memoria estable crece >10 %;
- carga/save empeora >15 % o cruza target;
- build size crece >10 %;
- aparecen allocations recurrentes nuevas;
- un spike nuevo supera 50/100 ms;
- aumenta el número de errores/warnings.

Los porcentajes son triggers de investigación, no fallos automáticos si el cambio es esperado y aprobado.

22.3. Pruebas automatizadas

Las pruebas no sustituyen el profiler, pero pueden detectar:

- escena y profile correctos;
- ausencia de roots duplicados;
- límites de población;
- no allocations en funciones puras mediante pruebas específicas;
- save size/tiempo con tolerancia amplia en entorno controlado;
- catálogos sin duplicados;
- materials/prefabs con reglas;
- LODGroup presente donde se exige;
- calidad y opciones válidas;
- performance markers disponibles en builds técnicas.

No se crean tests temporales frágiles dependientes de carga del CI sin calibración.

22.4. Pruebas manuales

- cámara y movimiento;
- build mode intensivo;
- ocho clientes;
- UI extrema;
- cierre/save/load;
- siete días;
- alt-tab;
- presets;
- hardware mínimo candidato;
- audio;
- thermal soak.

22.5. Player.log

Se revisa por:

- exceptions;
- warnings repetidos;
- missing refs;
- shader errors;
- audio missing;
- save retry;
- scene duplication;
- fallback inesperado;
- stack traces;
- logs por frame.

22.6. Informe de rendimiento

El informe contiene:

1. resumen ejecutivo;
 2. build/hardware;
 3. escenarios;
 4. targets;
 5. resultados;
 6. CPU/GPU;
 7. memoria/GC;
 8. carga/save;
 9. regresiones;
 10. defectos;
 11. deuda;
 12. decisión PASS/FAIL.
-

23. Gates por fase

23.1. Sprint 16

No exige optimización final, pero sí:

- StoreInitial representativa integrable;
- assets sin defectos técnicos graves;

- no regresión catastrófica;
- LOD/material/collider revisados;
- build posterior a integración;
- baseline inicial capturada;
- riesgos transferidos a Sprint 17.

23.2. Sprint 17

Obligatorio:

- Build de profiling;
- hardware registrado;
- escenarios 001-015;
- CPU/GPU/memoria;
- load/save;
- frame pacing;
- corrección de S0/S1;
- regresiones críticas cerradas;
- objetivos medidos;
- informe firmado.

23.3. H6

PASS solo si:

- 60 FPS/1080p target cumplido en referencia o desviación formalmente bloquea/reestructura;
- hasta ocho clientes;
- carga/save dentro de presupuesto o decisión documentada;
- sin fuga/crecimiento continuo;
- siete días estables;
- accesibilidad extrema operable;
- Player.log limpio;
- build identificada;
- evidencia archivada;
- no se han degradado arte, audio o claridad de forma inaceptable.

23.4. Demo/Playtest

Además:

- matriz de hardware ampliada;
- primera ejecución limpia;
- caps/presets;
- reportes de usuarios;
- diagnósticos privados y consentidos;
- requisitos provisionales etiquetados como tales.

23.5. Alpha/Beta

- hardware mínimo/recomendado candidatos;
- drivers representativos;
- contenido casi completo;
- memoria/build size;
- long soak;
- opciones finales;
- Steam Deck solo si entra en alcance;
- regressions automatizadas donde sea viable.

23.6. Release Candidate

- build release-like;
- requisitos publicados reproducibles;
- performance report final;
- S0/S1 = 0;
- S2 cerrados o aceptados;
- rollback/hotfix preparado;
- checksums;
- compatibilidad con save;
- capturas comerciales tomadas con ajustes declarados.

23.7. Postlanzamiento

- monitorizar crashes y reports;
- solicitar hardware/logs de forma privada;
- reproducir en build afectada;
- no recopilar hardware sin política/consentimiento;
- hotfix con benchmark antes/después;
- actualizar requisitos solo con evidencia.

24. Roles y responsabilidades

Área	Responsabilidad
Product/Design	define experiencia y qué no se sacrifica
Engineering	instrumenta, mide y corrige
Art	respetar presupuestos y valida calidad
UI/UX	controla rebuild, escalado y respuesta
Audio	voces, carga y streaming
QA	escenarios, repetibilidad y defectos
Production	reserva capacidad y gestiona gates
Build/Release	perfiles, artefactos y reproducibilidad
Marketing/Steam	no publica claims/requisitos sin PASS
Security/Privacy	protege captures, logs y datos de hardware

En un proyecto unipersonal una misma persona puede ocupar todos los roles, pero los artefactos y momentos de decisión deben mantenerse separados.

25. Registro inicial de deuda y hallazgos

ID	Hallazgo observado	Riesgo
PERF-DEBT-001	StoreInitial integrada en build de cierre 0.0.26	RESUELTA S16
PERF-DEBT-002	TestLab en Development Profile	no apto para release
PERF-DEBT-003	1024×768 por defecto	no representa target
PERF-DEBT-004	ventana no redimensionable	UX/PC
PERF-DEBT-005	VSyn 0 sin cap explícito	pacing/consumo
PERF-DEBT-006	no hardware de referencia cumplimentado	target no reproducible
PERF-DEBT-007	existe Player post-StoreInitial, pero falta baseline de profiling	rendimiento no aprobado

ID	Hallazgo observado	Riesgo
PERF-DEBT-008	ResolveReferences en Update	CPU/búsquedas
PERF-DEBT-009	Update por cliente	escalabilidad
PERF-DEBT-010	wall occlusion por LateUpdate	CPU/material state
PERF-DEBT-011	composición/stock con Instantiate/Destroy	spikes/GC
PERF-DEBT-012	búsquedas globales en fallbacks	coste/lifecycle
PERF-DEBT-013	SSAO y sombras costosas por defecto	GPU
PERF-DEBT-014	opaque/depth textures siempre requeridas	bandwidth
PERF-DEBT-015	streaming mipmaps desactivado	memoria futura
PERF-DEBT-016	personajes finales ausentes	baseline subestima coste
PERF-DEBT-017	VFX finales ausentes	baseline subestima overdr
PERF-DEBT-018	AudioMixer/streaming final ausentes	baseline incompleta
PERF-DEBT-019	no Memory Profiler baseline	fugas no demostradas
PERF-DEBT-020	no requisitos mínimos/recomendados	Steam bloqueado
PERF-DEBT-021	build size sin baseline	regresiones invisibles
PERF-DEBT-022	sin markers específicos de rendimiento	atribución limitada
PERF-DEBT-023	Quality presets no orientados a PC	escalabilidad limitada
PERF-DEBT-024	no informe de shader variants	hitch/build size

26. Work packages

PERF-WP-001 — Congelar benchmark de referencia

Objetivo: crear save, escena, cámara y secuencia reproducible.

Salida: Performance_Benchmark_Save, instrucciones y checksum.

Aceptación: otra ejecución reproduce población, layout y recorrido.

PERF-WP-002 — Registrar hardware DEV-REF

Completar CPU, GPU, RAM, disco, OS, driver, monitor, energía y fecha.

PERF-WP-003 — Crear Windows_Profiling

Perfil con StoreInitial, Development sin Deep Profile por defecto, escenas correctas y metadatos.

PERF-WP-004 — Crear Windows_ReleaseCandidate

Perfil release-like sin TestLab ni tooling técnico.

PERF-WP-005 — Política de frame pacing

Implementar y probar VSync/caps, persistencia y defaults.

PERF-WP-006 — Opciones de resolución/ventana

Borderless/windowed/fullscreen, resize, confirmación y rollback.

PERF-WP-007 — Presets PC

Diseñar Low/Medium/High y documentar diferencias medibles.

PERF-WP-008 — Instrumentación base

Markers para escena, customers, UI, placement, save/load, day close y audio.

PERF-WP-009 — Baseline CPU/GPU StoreInitial

Capturas de escenarios vacío/equipado/ocho clientes.

PERF-WP-010 — Baseline memoria

Snapshots MainMenu, StoreInitial, ocho clientes, siete días y retorno.

PERF-WP-011 — Baseline carga

Cold boot, nueva partida, scene load y load slot.

PERF-WP-012 — Baseline save

Desglosar serialización, checksum, I/O, verificación y replace.

PERF-WP-013 — Auditoría Updates

Medir once métodos por frame, frecuencia, coste y allocations.

PERF-WP-014 — Customer scheduling

Determinar si Update por agente cumple a 8 y stress; introducir scheduler solo si falla.

PERF-WP-015 — Navegación

Capturar path requests, fallos, reintentos y cambios de layout.

PERF-WP-016 — Placement

Perf de preview, raycast, occupancy, material y commit.

PERF-WP-017 — Inventory visual sync

Medir instanciación, agrupación de productos y cambios de stock.

PERF-WP-018 — UI profiling

Canvas rebuild, layout, HUD, Operations, listas, ES/EN y 150 %.

PERF-WP-019 — Shadow study

A/B de distancia, cascadas, resolución, soft y additional shadows.

PERF-WP-020 — SSAO study

A/B visual/coste y mapping por preset.

PERF-WP-021 — Opaque/depth audit

Identificar consumers y eliminar requisitos no usados si procede.

PERF-WP-022 — Material/batch audit

SetPass, materiales únicos, SRP Batcher y MaterialPropertyBlock.

PERF-WP-023 — LOD/culling validation

Popping, ahorro y bounds en escena real.

PERF-WP-024 — Texture budget

Definir import presets antes de producción de texturas.

PERF-WP-025 — Character budget

Tras integrar modelos, medir Animator, skinning, materiales y ocho clientes.

PERF-WP-026 — VFX budget

Partículas, overdraw, pooling y reduced motion.

PERF-WP-027 — Audio profiling

Voces, CPU, memoria, streaming, loops y spam.

PERF-WP-028 — Scene lifecycle soak

Diez ciclos y detección de duplicados/retenciones.

PERF-WP-029 — Seven-day soak

Golden Path largo, memoria, save size, timings y logs.

PERF-WP-030 — Hardware mínimo candidato

Seleccionar, ejecutar matriz y decidir 30/60/quality.

PERF-WP-031 — Hardware recomendado candidato

Validar 1080p60 con calidad aprobada.

PERF-WP-032 — Build size/shader report

Baseline y variantes antes de RC.

PERF-WP-033 — Performance regression report

Plantilla y proceso por hito.

PERF-WP-034 — H6 Performance Report

Informe consolidado con decisión PASS/FAIL.

PERF-WP-035 — Steam requirements approval

Traducir evidencia a mínimos/recomendados sin claims excedidos.

PERF-WP-036 — CI/performance smoke

Solo tras disponer de máquina estable: scene load, timings amplios y report, evitando gates flakey.

PERF-WP-037 — Postlaunch diagnostics

Flujo de recopilación manual/opt-in compatible con privacidad.

27. Registro de riesgos

ID	Riesgo	Prob.	Impacto	Mitigación
PERF-RSK-001	optimizar Editor y fallar Player	alta	alta	profiling en build
PERF-RSK-002	hardware potente oculta problemas	alta	alta	candidatos mínimo/re
PERF-RSK-003	StoreInitial no representativa	alta	crítica	gate S16 antes de m
PERF-RSK-004	personajes finales elevan CPU/GPU	alta	alta	repetir baseline tras i
PERF-RSK-005	texturas/VFX elevan memoria/overdraw	alta	alta	presupuestos de imp
PERF-RSK-006	frame cap indefinido	alta	media/alta	política explícita
PERF-RSK-007	sombras/SSAO dominan GPU	media	alta	presets y A/B
PERF-RSK-008	Update por agente escala mal	media	alta	profiling y scheduler
PERF-RSK-009	NavMesh spikes tras layout	media	alta	path policy, no rebak
PERF-RSK-010	save bloquea cierre	media	crítica	markers, feedback, o
PERF-RSK-011	pooling introduce estado residual	media	alta	reset contract/tests
PERF-RSK-012	caches causan leak	media	alta	límites y lifecycle
PERF-RSK-013	UI extrema rompe frame time	media	alta	perfil ES/EN 150 %
PERF-RSK-014	logs causan stutter	media	media	rate limit y build leve
PERF-RSK-015	shader hitch en primer uso	media	alta	variants/warm-up me
PERF-RSK-016	quality Low rompe legibilidad	media	alta	review art/accessibili
PERF-RSK-017	VSync/cap causa pacing irregular	media	alta	hardware/refresh mar
PERF-RSK-018	thermal throttling en portátil	media	media/alta	soak térmico
PERF-RSK-019	requisitos de Steam demasiado optimistas	media	crítica	no publicar sin RC
PERF-RSK-020	cambios de paquetes invalidan baseline	media	alta	benchmark por upgra
PERF-RSK-021	CI performance flakey	alta	media	no gatear sin máquin
PERF-RSK-022	optimización rompe determinismo	media	crítica	tests de invariantes
PERF-RSK-023	optimización elimina feedback	media	alta	review UX/accessibili
PERF-RSK-024	Addressables prematuros añaden complejidad	media	media	decisión por necesida
PERF-RSK-025	Memory Profiler snapshots no comparables	media	media	mismo build/escenari

28. Playbooks operativos

28.1. Caída repentina de FPS

1. confirmar build/hardware/configuración;
2. reproducir escenario;
3. comparar commit anterior;
4. determinar CPU/GPU;
5. revisar logs;
6. bisect o desactivar subsistemas controladamente;
7. corregir causa;

8. repetir tres pasadas;
9. ejecutar regresión funcional;
10. registrar resultado.

28.2. Stutter al abrir panel

- marker de apertura;
- UI Profiler/Timeline;
- comprobar instanciación, layout, fuente, Resources, strings y GC;
- precargar solo recursos necesarios;
- reutilizar vistas si aporta;
- verificar foco y accesibilidad.

28.3. Crecimiento de memoria

- snapshot A/B;
- repetir acción;
- forzar unload/GC solo como diagnóstico;
- buscar referencias estáticas, eventos, DontDestroy, pools y assets;
- comprobar retorno a MainMenu;
- corregir owner/lifecycle;
- soak de confirmación.

28.4. GPU bound

- confirmar con GPU time;
- variar resolución;
- shadows;
- SSAO;
- overdraw;
- postproceso;
- materials/SetPass;
- LOD/culling;
- crear cambio de preset o asset;
- validar calidad.

28.5. CPU bound por clientes

- separar lógica, path, animation y presentation;
- marker por fase;
- medir por agente;
- reducir búsquedas;
- event/tick scheduling;
- batch de decisiones;
- stress;
- verificar resultados.

28.6. Save lento

- medir fases;
- tamaño del snapshot;
- strings/listas;
- checksum/I/O;
- disco;
- antivirus;
- optimizar sin retirar atomicidad;
- feedback;
- comparar save pequeño/grande.

28.7. Load lento

- separar I/O, parse, migration, restore, visual sync y scene activation;
- detectar Resources o catálogos repetidos;
- batch visual;
- async segura;
- loading state;
- cold/warm comparison.

28.8. Shader hitch

- reproducir primera aparición;
- identificar shader/variant;
- revisar feature innecesaria;
- stripping;
- warm-up selectivo;
- evitar material creado tardíamente;
- validar build size.

28.9. Build peor que Editor o viceversa

- comparar quality/profile;
- Development flags;
- resolution;
- VSync/cap;
- scripting/stripping;
- logs;
- driver/GPU selection;
- scene list;
- assets incluidos.

28.10. Report de usuario

Solicitar de forma privada y proporcional:

- versión/build;
- OS;
- CPU/GPU/RAM;
- resolución/preset;
- escenario;
- log;
- vídeo opcional;
- save solo con consentimiento y copia.

No pedir información personal innecesaria.

29. Plantillas de evidencia

29.1. Cabecera de benchmark

Performance Run ID:
Build ID / SHA:
Unity:
Date:
Tester:
Hardware ID:
OS / Driver:

Resolution / Refresh:
Display Mode:
Quality / VSync / Cap:
Scene / Save:
Customers:
Duration / Warm-up:
Profiler mode:
Notes:

29.2. Resultado de escenario

Scenario ID:
Target:
Median frame time:
P95:
P99:
Frames >33.33 ms:
Max spike:
Main thread:
GPU:
GC Alloc:
Memory start/end/peak:
Load/save time:
Errors/warnings:
PASS / FAIL / DEBT:

29.3. Comparación antes/después

Campo	Antes	Después	Delta	Decisión
Mediana				
P95				
P99				
Main thread				
GPU				
GC Alloc				
Memory				
Load/save				
Calidad visual				
Riesgo				

29.4. Deuda aceptada

Debt ID:
Scenario:
Measured deviation:
User impact:
Reason accepted:
Mitigation:
Owner:
Trigger:
Review milestone/date:
Related tests/docs:

30. Checklists

30.1. Antes de medir

- ☐ Build y SHA registrados.
- ☐ Hardware y driver registrados.
- ☐ Escena/save congelados.
- ☐ Resolución, calidad, VSync y cap fijos.
- ☐ StoreInitial representativa.
- ☐ Población y layout correctos.
- ☐ Warm-up definido.
- ☐ Markers disponibles.
- ☐ Logs limpios de errores ajenos.
- ☐ Captura no usa Deep Profile salvo diagnóstico.

30.2. CPU

- ☐ Main thread.
- ☐ Render thread.
- ☐ Update/LateUpdate.
- ☐ customer tick.
- ☐ navigation.
- ☐ UI.
- ☐ placement.
- ☐ save/load.
- ☐ strings/logs.
- ☐ waits.

30.3. GPU

- ☐ GPU time.
- ☐ resolución.
- ☐ sombras.
- ☐ luces.
- ☐ SSAO.
- ☐ transparencias.
- ☐ postproceso.
- ☐ batches/SetPass.
- ☐ LOD/culling.
- ☐ variants/hitch.

30.4. Memoria

- ☐ MainMenu.
- ☐ StoreInitial.
- ☐ ocho clientes.
- ☐ save/load.
- ☐ siete días.
- ☐ retorno a menú.
- ☐ managed/native.
- ☐ textures/meshes/audio.
- ☐ pools/caches.
- ☐ crecimiento explicado.

30.5. UI/accesibilidad

- ☐ ES.
- ☐ EN.
- ☐ pseudolocale.
- ☐ UI 150 %.

- ☐ texto 150 %.
- ☐ resolución mínima.
- ☐ reducción de movimiento.
- ☐ teclado/ratón.
- ☐ mando cuando entre en alcance.
- ☐ foco y modal.

30.6. H6

- ☐ 1080p60 target medido.
 - ☐ ocho clientes.
 - ☐ frame pacing.
 - ☐ carga <5 s o decisión.
 - ☐ save/cierre <2 s o decisión.
 - ☐ allocations estables.
 - ☐ sin fuga.
 - ☐ siete días.
 - ☐ Player.log.
 - ☐ informe y evidencias.
 - ☐ S0/S1 = 0.
 - ☐ deuda aprobada.
-

31. Integración con otros documentos

31.1. Vertical Slice Specification

Actualizar tras Sprint 17 con hardware de referencia, percentiles y resultado real. No sustituir el target histórico sin Change ID.

31.2. TDD

Añadir decisiones de scheduling, quality, pooling o async que se implementen. Este plan no debe convertirse en duplicado de clases.

31.3. QA Plan/Matrix

Crear casos PERF-SCN y columnas de build, hardware, percentiles, memoria y evidencia.

31.4. Art Bible

Actualizar presupuestos solo con profiling. Registrar excepciones por asset y presets visuales.

31.5. Audio Bible

Registrar load type, voice limits y resultados de Audio Profiler.

31.6. UI Style Guide y Accessibility

Alinear escalado, presets, reducción de movimiento, resolución mínima y respuesta.

31.7. Build Guide

Formalizar Windows_Profiling, RC y metadata de benchmark.

31.8. Steam Publishing y Marketing

No publicar:

- “optimizado”;
- “funciona en equipos modestos”;
- requisitos mínimos/recomendados;
- Steam Deck Verified/Playable;
- 60 FPS general;

hasta disponer de evidencia correspondiente.

31.9. Privacy y Security

Logs, hardware records, profiler captures y saves de usuarios se almacenan con minimización, acceso restringido y retención definida. Builds con símbolos o profiler no se distribuyen públicamente.

31.10. Documentos 31-34

- 31_H6_and_Release_Readiness_Checklist.xlsx incluirá gates y evidencias de este plan.
- 32_Documentation_Baseline_Manifest.xlsx registrará versión/hash del documento 30 y del informe final.
- 33_Project_Risk_Register_and_Business_Continuity_Plan.xlsx absorberá riesgos globales de hardware, builds y herramientas.
- 34_Final_Handoff_and_Project_Operations_Manual.md explicará cómo repetir benchmarks.

32. Fuentes históricas directas

La tabla siguiente conserva todas las versiones Markdown localizadas de las familias documentales con impacto directo en rendimiento. La inclusión de una fuente no implica que cada afirmación siga vigente; permite distinguir evolución, reedición y sustitución.

```
|---|---:|---|
```

33. Fuentes vigentes y artefactos inspeccionados

33.1. Documentación consolidada

Se ha revisado el conjunto:

- 00_Enfoque_y_Alcance.md;
- 01_Game_Design_Document.md;
- 02_Vertical_Slice_Specification.md;
- 03_Technical_Design_Document.md;
- 04_Modelo_de_Datos.md;
- 05_UX_Flow.md;
- 06_Production_Roadmap_y_Sprint_Plan.md;
- 07_QA_Testing_Plan.md;
- 08_QA_Testing_Matrix.xlsx;
- 09_CSharp_Coding_Standards.md;
- 10_Unity_Project_Setup_Guide.md;
- 11_Build_y_Versioning_Guide.md;
- 12_Excel_Maestro_de_Produccion.xlsx;
- 13_Trazabilidad_y_Control_de_Cambios.xlsx;
- 14_Project_Binder_Indice_Maestro.md;

- 15_Guia_Maestra.md;
- 16_Auditoria_Global_de_Coherencia.md;
- 17_Art_Bible.md;
- 18_Audio_Bible.md;
- 19_UI_Style_Guide.md;
- 20_Economy_and_Balance_Specification.md;
- 21_Initial_Content_Catalog.xlsx;
- 22_Localization_Plan.md;
- 23_Legal_Credits_and_Licenses_Register.xlsx;
- 24_Steam_Publishing_Plan.md;
- 25_Post_Launch_and_Live_Operations_Plan.md;
- 26_Privacy_Data_and_Telemetry_Plan.md;
- 27_Security_and_Incident_Response_Plan.md;
- 28_Marketing_and_Communication_Plan.md;
- 29_Accessibility_and_Inclusive_Design_Plan.md.

33.2. Configuración inspeccionada

- ProjectSettings/ProjectVersion.txt;
- ProjectSettings/ProjectSettings.asset;
- ProjectSettings/QualitySettings.asset;
- Packages/manifest.json;
- Packages/packages-lock.json cuando corresponde;
- Assets/_Project/Settings/PC_RPAsset.asset;
- Assets/_Project/Settings/PC_Renderer.asset;
- Assets/_Project/Settings/DefaultVolumeProfile.asset;
- Assets/_Project/Settings/BuildProfiles/Windows_Development.asset;
- escenas Bootstrap, MainMenu, Store, StoreInitial y TestLab;
- scripts runtime, presentation, infrastructure, editor y tests;
- prefabs, FBX, materials, audio, animaciones y catálogos.

33.3. Registros históricos de desarrollo

Se han considerado ADR, charters, cierres, checklists, matrices y handoffs de Sprints 0-17, especialmente:

- scene flow y Build Profiles;
- input/cámara;
- placement y access validation;
- customers/spawning;
- save/load;
- UI/UX Sprint 15;
- integración representativa Sprint 16;
- preparación de estabilización Sprint 17.

33.4. Limitación de la fotografía

El proyecto inspeccionado corresponde al estado previo a la regeneración documental. Este plan no afirma que una rama futura conserve exactamente los mismos ajustes. Antes de ejecutar work packages se verificará branch, commit, Unity, paquetes, escenas y build profile.

34. Criterios de cierre del documento 30

El documento 30 se considera generado cuando:

1. conserva los presupuestos v0.3/v0.4 y no los borra por las versiones breves posteriores;

2. integra la fotografía v0.5/v0.6 y el estado real de Unity;
3. distingue target, configuración observada y evidencia pendiente;
4. define frame time, percentiles, CPU, GPU, memoria, GC, carga y save;
5. cubre render, arte, UI, clientes, navegación, audio, input y build;
6. establece una matriz de hardware sin inventar requisitos publicables;
7. define escenarios, metodología, herramientas, markers y storage;
8. establece gates de Sprint 16, Sprint 17, H6, demo, beta y RC;
9. documenta deuda, riesgos, work packages y playbooks;
10. protege determinismo, integridad de save, accesibilidad y claridad;
11. impide claims públicos sin benchmark;
12. puede alimentar los documentos operativos 31-34 y la auditoría final.

Estado de este documento: COMPLETE AS CONSOLIDATED PLAN / MEASUREMENT, IMPLEMENTATION AND RELEASE VALIDATION PENDING.

Siguiente documento de la secuencia: 31_H6_and_Release_Readiness_Checklist.xlsx.

Actualización de rendimiento W0

Estado operativo vigente tras W0

Elemento	Estado vigente
Baseline de entrada	main@efbc1a885a1d6819f764ec8cff8a465ad1986661 / aplicación 0.0.26
W0 documental	COMPLETED / DOCUMENTAL PASS
Decisiones funcionales abiertas	0
Sprint17_Phase1	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
H6	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
Vertical Slice	ACCEPTED / IMPLEMENTED / VALIDATED / CLOSED

Escenarios de caracterización obligatorios

- Día de 300 s a x0,5, x1, x2 y x4 con equivalencia lógica dentro de la tolerancia definida.
- Pausa sostenida y anidada sin avance de spawn, paciencia, cierre, entregas o impuestos.
- Ocho clientes activos recorriendo browsing, reserva, cola, checkout, abandono y salida.
- Process All con varias órdenes y recepción sin duplicados ni allocations crecientes por rein-tento.
- Campaña persistente de siete días con histórico, impuesto semanal, save/reload y comparación de memoria.
- Wall occlusion efectiva false; el perfil no puede justificar reactivar una opción descartada por contrato.
- Regresión de pivotes/GroundAnchor sin cambios de transforms o navegación tras load.

El objetivo de W0 es congelar el escenario, no declarar métricas. Todos los resultados permanecen APPROVED / VALIDATED tras W1-W8.

Regla de cierre y no propagación

- W0-W8 están completadas, validadas y cerradas en la baseline 0.0.26.
- Sprint17_Phase1 y W8 están completadas, validadas y cerradas.
- H6 y la Vertical Slice disponen de aceptación propia y están cerrados sobre 0.0.26.
- La Vertical Slice queda cerrada; los sistemas Post-H6 se gestionarán como evolución posterior mediante control de cambios.
-